

SehatGuru

Kaavish Report
presented to the academic faculty
by

Sameer Kamani sk08109
Arsal Jangda aj08514
Hammad Malik hm08298
Ahtisham Uddin au08429



Habib University
s h a p i n g f u t u r e s

In partial fulfillment of the requirements for
Bachelor of Science
Computer Science

Dhanani School of Science and Engineering

Habib University
Spring 2026

SehatGuru

This Kaavish project was supervised by:

Dr. Abdul Samad
Faculty of Computer Science
Habib University

Approved by the Faculty of Computer Science on _____.

Dedication

This project is wholeheartedly dedicated to our beloved parents, who have raised and nurtured us with constant love, patience, and unwavering support throughout our four-year journey at Habib University. A special tribute to our parents for their endless sacrifices, prayers, and for supporting our dreams and career ambitions; we are truly nothing without your guidance. To our siblings and friends, who provided the encouragement and inspiration necessary to complete this endeavor and bolstered our spirits when we were struggling.

We also dedicate this project to the health-conscious individuals and the broader community in Pakistan, whose nutritional needs and cultural preferences inspired us to create **Sehat Guru**. It is our sincere hope that this solution serves as a meaningful companion in making their everyday lives healthier and more empowered.

Most importantly, the project is dedicated to the spirit and perseverance of our group: **SehatGuru**. Everyone sincerely supported and encouraged each other during tough times and spent countless hours of work to ensure the project's success.

Lastly, we dedicate this project to the Almighty God. Thank you for the guidance, strength, power of mind, and the skills provided to us to navigate this journey. All our success, we owe to You.

Acknowledgements

Team **Sehat Guru** would like to extend our heartfelt gratitude to our advisor, **Dr. Abdul Samad**, for giving us the opportunity to complete our Final Year Project under his expert guidance. His specialized knowledge in Artificial Intelligence and his sound counsel were invaluable in helping us achieve our technical objectives, particularly in the development of our food recognition and chatbot systems. It was a great privilege and honor to work and study under his mentorship.

We would like to express our gratitude to our friends, family, and the Habib University community for their contributions to the project, particularly those who participated in data gathering and the verification of traditional Pakistani dishes. Their genuine excitement for Sehat Guru and the underlying concept of culturally-aware nutrition tracking kept us motivated throughout.

We are extremely grateful to the Computer Science program, the Kaavish Working Group, and Habib University for providing us with the technological support and resources required for this study. Our many thanks to all the CS faculty members and professors at Habib University; they have truly shaped us into the individuals we are today, instilling in us a sense of excellence and social responsibility.

Finally, we wish to thank the Kaavish evaluators for their valuable feedback, which allowed us to strive for perfection in our design and implementation. We truly appreciate the collaborative environment that has brought this project to fruition.

Abstract

While digital health and nutrition tracking solutions have seen a global surge, their adoption in South Asian markets—particularly in Pakistan—remains limited due to significant cultural and dietary disparities. Most existing applications rely on Western-centric nutritional databases that lack representation of local cuisine and traditional eating habits. This project proposes **Sehat Guru**, an AI-powered nutrition companion specifically designed for the Pakistani context. The application provides an intuitive platform for users to manage their health goals through culturally-aware dietary tracking and personalized guidance.

Sehat Guru utilizes a multi-modal approach to simplify nutritional logging. Firstly, the work explores the development of a specialized image classification pipeline to recognize a subset of traditional Pakistani dishes, enabling users to log meals through visual identification. Secondly, the project integrates a contextual AI chatbot designed to engage with users in a human-like manner, providing personalized health advice and answering nutritional queries. To ensure precision, a unified nutritional calculation algorithm is implemented to estimate macronutrients and calories tailored to the local diet. By bridging the gap between advanced artificial intelligence and regional cultural nuances, Sehat Guru offers a novel solution for health management, laying a foundation for more localized e-health applications in the South Asian landscape.

Contents

Dedication	2
Acknowledgements	3
Abstract	4
1 Introduction	12
1.1 Problem Statement	12
1.2 Proposed Solution	12
1.3 Intended User	13
1.4 Project gantt chart and deliverables	13
1.5 Key Challenges	13
2 Literature Review	14
2.1 Literature Review	14
2.2 Review of Modern Nutrition Tracking Applications	14
2.2.1 Core Functionalities	15
2.2.2 Differentiating Features and Regional Adaptation	15
2.2.3 System Limitations	16
2.2.4 Comparative Summary of Applications	17
2.3 Integration of Large Language Models and Conversational Nutrition Systems	17
2.4 Food Classification for Dietary Automation	18
2.4.1 Lightweight Architectures for Mobile Deployment	18
2.4.2 Fine-Grained Recognition in Regional Contexts	18
2.4.3 Hybrid Architectures for Global Feature Modeling	19
2.4.4 From Recognition to Nutritional Estimation	19
2.4.5 Summary of Findings	20

2.5	Synthesis and Gap Analysis	20
2.5.1	Application Gap: Lack of Regional Adaptation	20
2.5.2	Data Gap: Bias in Visual Recognition	20
2.5.3	Context Gap: Reliability of AI Systems	21
2.6	Motivation for SehatGuru	21
3	Software Requirement Specification (SRS)	22
3.1	Functional Requirements	22
3.1.1	User Authentication and Profile Management	22
3.1.2	User Profile and Health Goal Setup	23
3.1.3	Nutritional Database	23
3.1.4	Food Logging (Camera Identification)	24
3.1.5	Food Logging (Manual Search)	24
3.1.6	Portion Size Selection	24
3.1.7	Nutritional Calculation	25
3.1.8	Food Recognition and Classification Implementation Details	25
3.1.9	AI Chatbot and Nutritional Assistance	25
3.1.10	Meal Planning and Meal Prep	26
3.1.11	AI Memory and Personalization	26
3.1.12	Progress Dashboard and Analytics	27
3.1.13	Personalized Notifications	27
3.1.14	Security and Data Management	27
3.1.15	User Feedback and Reviews	28
3.2	Non-functional Requirements	28
3.2.1	Performance	28
3.2.2	Security	28
3.2.3	Reliability	29
3.2.4	Usability	29
3.2.5	Compliance	30
3.2.6	Data Integrity	30
3.2.7	Maintainability	31
3.2.8	Availability and Rate Limiting	31
3.2.9	Observability	31
3.3	External Interfaces	31
3.3.1	User Interfaces	32
3.3.2	Application Program Interface (API)	36
3.3.3	Hardware/Communication Interfaces	36
3.4	Use Cases	36

3.5	Datasets	38
3.6	System Diagram	39
4	Software Design Specification (SDS)	40
4.1	Software Design	40
4.1.1	System Architecture	40
4.1.2	UML Class Diagram	42
4.1.3	Class Design Details	42
4.2	Data Design	46
4.2.1	Overview	46
4.2.2	Entity-Relationship Diagram	47
4.2.3	Database Schema (Normalized Relational Model)	48
4.2.4	UML-to-Database Mapping	49
4.2.5	Entity Relationships	49
4.2.6	Design Justification	50
4.3	Technical Details	51
5	Dataset	52
5.1	Dataset	52
5.1.1	Selection of Top 20 Pakistani Dishes	52
5.1.2	Image Dataset Curation	53
5.1.3	Nutritional Database Synthesis	56
6	Dish Classification Model Training	58
6.1	Model Selection and Architectures	58
6.1.1	EfficientNet-B0: Enhanced Baseline Development	58
6.1.2	Swin Transformer: Hierarchical Vision via Shifted Windows	62
6.1.3	ConvNeXt: Modernizing Convolutional Neural Networks	65
6.2	Training Methodology	68
6.3	Results and Comparative Analysis	69
7	Application	70
7.1	Register, Login and Onboarding	70
7.2	Main menu	72
7.3	Adding a Meal	73
7.4	Chatbot	76
7.5	Profile & Feedback	78
7.6	User Analytics dashboard	79

8	Methodology, Experiments and Results OR Testing and Analysis	81
9	Conclusion and Future Work	82
10	Reflection	83
	Appendix A More Math	84
	Appendix B Data	85
	Appendix C Code	86

List of Figures

1.1	Shows the risk factor of diseases linked to and nutrition in Pakistan [Lancet2023]	12
2.1	Healthify & My Fitness Pal App	15
2.2	An overview design of the classification	19
3.1	Login and Password Recovery Screens	32
3.2	Signup, Terms, and Email Verification Screens	32
3.3	User Profile Setup and Goal Selection Screens	33
3.4	Main Dashboard and Nutritional Breakdown Screens	33
3.5	Progress and Analytics Overview Screen	34
3.6	Food Logging Flow: Capture, Search, and Portion Selection Screens .	35
3.7	Chatbot, Meal Planning, Profile, and Account Management Screens .	35
3.8	Use Case Diagram for SehatGuru	37
3.9	System Block Diagram of SehatGuru	39
4.1	System Architecture Diagram	41
4.2	UML Class Diagram for Sehat Guru Application	42
4.3	Entity-Relationship Diagram for SehatGuru Database	47
5.1	Class distribution of the primary training dataset (5,000+ images). .	54
5.2	Visual samples of cleaned and cropped food images used for model training.	54
5.3	Class distribution of the manually curated real-world validation dataset.	55
5.4	Sample images from the validation set illustrating real-world lighting and diverse plating.	56
5.5	Visual representation of the nutritional database composition and source distribution.	57

6.1	Comparison of standard scaling methods vs. EfficientNet’s Compound Scaling logic [tan2019efficientnet].	59
6.2	Confusion Matrix for EfficientNet-B0 on the Test Set, highlighting misclassifications in similar-looking gravy dishes.	61
6.3	Prediction visualization demonstrating the average validation confidence of 65% across classified dishes.	62
6.4	Architecture of the Swin Transformer highlighting the hierarchical stages and shifted window self-attention [liu2021swin].	63
6.5	Confidence visualization for Swin Transformer, showing a marked improvement in prediction certainty (87%).	64
6.6	Confusion Matrix for Swin Transformer on the Test Set.	65
6.7	The architectural evolution of ConvNeXt, demonstrating the transition from standard ResNet blocks to Transformer-inspired layers [wang2025econvnext].	66
6.8	Confusion Matrix for the optimized ConvNeXt architecture on the Test Set.	67
6.9	Final prediction visualization showing consistent 88% average confidence across South Asian food categories.	68
7.1	User Registration and Onboarding Flow	71
7.2	Main Dashboard displaying daily calorie and macronutrient tracking .	73
7.3	Food Logging via Image Recognition and Manual Input	75
7.4	AI Chatbot Interface providing personalized nutrition assistance . . .	77
7.5	User Profile interface showing personal information and settings . . .	79
7.6	User Analytics Dashboard showing trends and behavioral insights . .	80

List of Tables

2.1	Comparison of leading nutrition tracking applications.	17
2.2	Comparison of key literature for mobile food recognition.	20
4.1	SehatGuru Database Schema	48
4.2	UML Class to Database Table Mapping	49
4.3	Entity Relationships Summary	50
5.1	Frequency Distribution of Top 20 Identified Dishes	53
6.1	Comparative Performance of Evaluated Architectures	69

1. Introduction

1.1 Problem Statement

This section explains the problem we are solving and our motivation to do so.

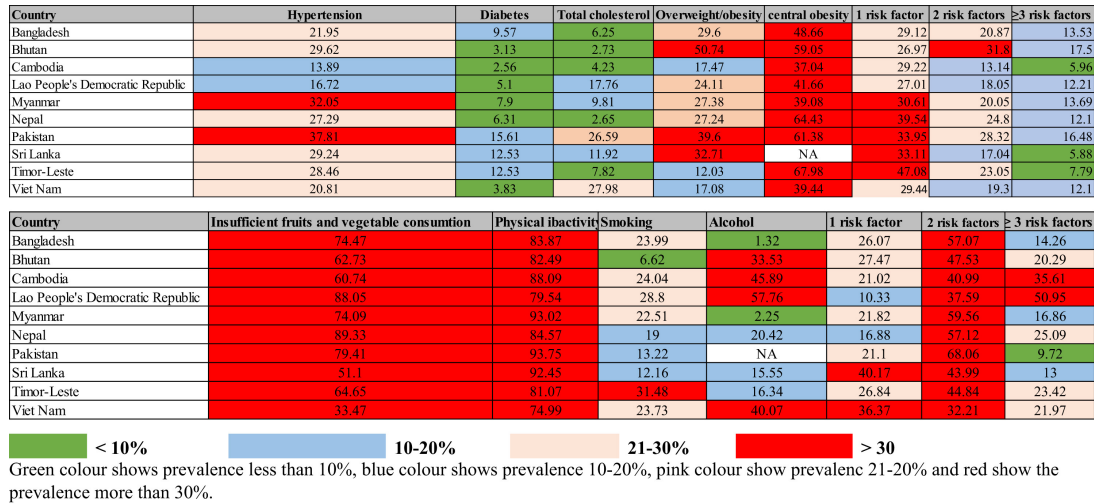


Figure 1.1: Shows the risk factor of diseases linked to and nutrition in Pakistan [Lancet2023]

1.2 Proposed Solution

This section gives a summary of our proposed solution, i.e. how does it solve the problem? An overview of the system is provided. A detailed description of each module of the system is presented later in Chapter 1.

1.3 Intended User

This section outlines the target users of this system. The different types of users in our user base and their interaction with the system are described briefly.

1.4 Project gantt chart and deliverables

Please include a detailed gantt and details of the deliverables for Kaavish I and II

1.5 Key Challenges

This section mentions the key challenges that we foresee in this project and possible ways to address them.

2. Literature Review

2.1 Literature Review

This section reviews existing work across three key dimensions relevant to SehatGuru: modern nutrition tracking applications, the integration of Large Language Models (LLMs) in healthcare & nutrition systems, and machine learning approaches for food classification. The review evaluates current capabilities, identifies technical and practical limitations, and highlights gaps in automation, cultural adaptation, and system reliability. These insights form the basis for the design decisions underlying SehatGuru.

2.2 Review of Modern Nutrition Tracking Applications

Modern mobile health applications such as MyFitnessPal [**myfitnesspal2024**], HealthifyMe [**healthifyme2024**], and YAZIO [**yazio2024**] represent the current state of digital nutrition tracking. These platforms provide mechanisms for caloric and macronutrient monitoring, dietary logging, and personalized recommendations. However, despite their widespread adoption, they primarily rely on structured user input and are optimized for global audiences rather than region-specific dietary contexts.

According to the World Health Organization [**WHO2023**], the rising burden of non-communicable diseases in Pakistan highlights the need for accessible and culturally relevant digital health interventions. Evaluating the strengths and limitations of these applications reveals critical design considerations for improving automation, usability, and localization.

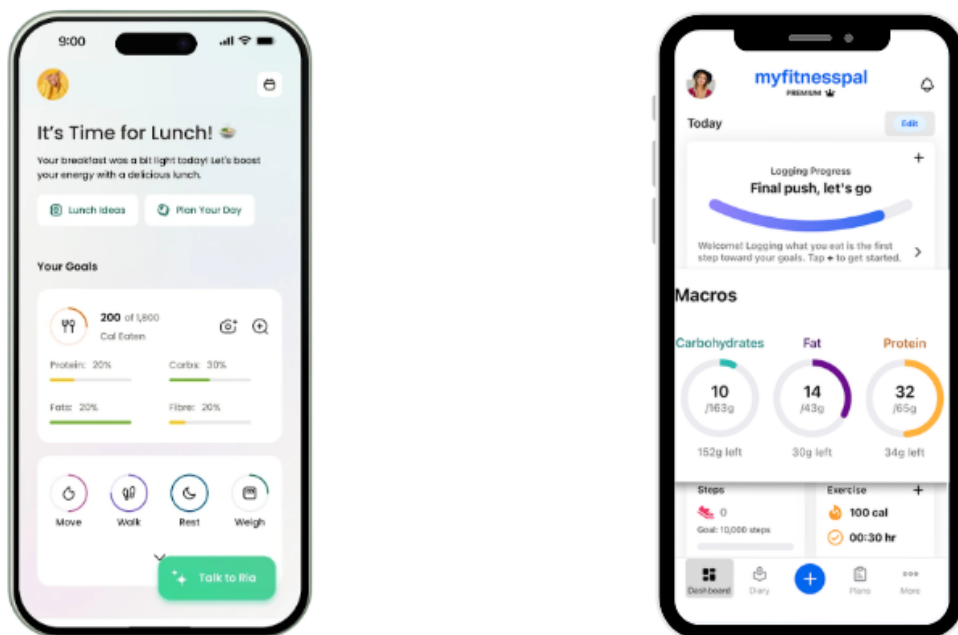


Figure 2.1: Healthify & My Fitness Pal App

2.2.1 Core Functionalities

Most contemporary nutrition tracking applications share a common functional foundation. These systems require users to manually log meals and track caloric and macronutrient intake, including fats, carbohydrates, and protein. During onboarding, demographic and physiological data such as age, gender, height, weight, and activity level are collected to estimate daily caloric requirements and generate personalized recommendations.

Additional features such as hydration tracking and basic activity monitoring are also widely supported. Integration with external health ecosystems, including Fitbit, Garmin, and Samsung Health, enables automatic synchronization of activity data. However, these integrations primarily enhance fitness tracking rather than reducing the core burden of dietary logging [myfitnesspal2024, healthifyme2024].

2.2.2 Differentiating Features and Regional Adaptation

While these applications share a similar foundation, they diverge in their approaches to automation and personalization. MyFitnessPal supports multiple logging modalities, including barcode scanning, voice input, and image-based meal recognition

through its premium Meal Scan feature [myfitnesspal2024]. However, its food database is largely Western-centric, limiting its effectiveness for non-Western cuisines.

HealthifyMe attempts to address this limitation by incorporating South Asian dietary patterns. Its Snap feature enables image-based food recognition, while its AI assistant Ria provides personalized dietary guidance [healthifyme2024][googleplay2024]. Despite this regional focus, its automation capabilities remain constrained by the need for user verification.

YAZIO emphasizes structured dieting strategies such as intermittent fasting and goal-oriented meal planning, supplemented by social accountability features. However, it lacks meaningful support for culturally diverse food datasets [yazio2024].

2.2.3 System Limitations

Despite advancements in functionality, these applications exhibit consistent limitations. Manual meal logging remains the primary interaction mechanism, requiring users to estimate portion sizes and validate nutritional information. This process introduces both cognitive burden and potential inaccuracies, particularly for complex, multi-ingredient dishes common in Pakistani cuisine.

Automated tools such as image recognition provide only partial support and often fail in real-world scenarios involving mixed or visually similar foods, necessitating manual correction [laing2014]. Additionally, many advanced features—including AI-driven recommendations and detailed analytics—are restricted to premium tiers, limiting accessibility for a broader user base.

2.2.4 Comparative Summary of Applications

Application	Automation Level	Cultural Adaptation	Key Limitation
MyFitnessPal	Moderate (barcode, image scan)	Low (Western-focused database)	Poor handling of regional dishes
HealthifyMe	Moderate (AI assistant + image scan)	Medium (South Asian support)	Requires user verification for accuracy
YAZIO	Low (manual logging focused)	Low	Limited automation and dataset diversity

Table 2.1: Comparison of leading nutrition tracking applications.

While these platforms demonstrate the feasibility of digital nutrition tracking, they collectively fail to deliver a seamless, automated, and culturally adaptive experience.

2.3 Integration of Large Language Models and Conversational Nutrition Systems

While traditional applications rely on structured input and static interfaces, recent advances in artificial intelligence have introduced conversational systems capable of interactive dietary guidance. Large Language Models (LLMs) enable natural interaction and personalized recommendations, shifting the paradigm from passive tracking to active coaching.

However, the deployment of LLMs in healthcare introduces significant reliability challenges. Thirunavukarasu et al. [thirunavukarasu2023] demonstrate that while models such as GPT-4 exhibit strong reasoning and conversational abilities, they are prone to generating hallucinated or unverified medical information when operating without external grounding.

Similarly, Belkhouribchia and Pen [belkhouribchia2025] highlight that although LLMs are effective for personalization, their susceptibility to factual inaccuracies remains a major barrier to clinical adoption. These findings indicate that standalone LLMs are insufficient for safety-critical applications such as nutrition guidance.

To address this limitation, Retrieval-Augmented Generation (RAG) architectures integrate LLMs with structured knowledge bases. This approach enables the model to retrieve verified information before generating responses, thereby improving factual accuracy while retaining conversational flexibility. Compared to pure fine-tuning approaches, RAG systems provide better control over data sources, reduce hallucination risk, and allow domain-specific customization without retraining large models.

For SehatGuru, this architectural paradigm enables the use of LLMs as reasoning engines grounded in a verified Pakistani food database, ensuring both contextual relevance and nutritional reliability.

2.4 Food Classification for Dietary Automation

Automating dietary tracking requires accurate recognition of food items from images, particularly in real-world scenarios involving visually similar dishes such as *Chapati* and *Naan*. This task falls under fine-grained image classification, where subtle visual differences must be distinguished under constraints of latency and computational efficiency.

2.4.1 Lightweight Architectures for Mobile Deployment

Recent research prioritizes efficient architectures suitable for mobile environments. Philip [philip2024] demonstrates that **EfficientNetB0** achieves strong performance on Food-101 [bossard2014] while maintaining computational efficiency. Similarly, Patel and Modi [patel2023] show that **MobileNetV3**, designed specifically for resource-constrained devices, achieves high accuracy on South Asian food datasets when combined with transfer learning and data augmentation.

These findings establish lightweight CNNs as practical candidates for real-time mobile deployment.

2.4.2 Fine-Grained Recognition in Regional Contexts

Recognizing regional cuisines introduces additional complexity due to high intra-class similarity. Gull et al. [gull2024] address this challenge through **SAFC_net**, a modified MobileNet architecture that enhances feature extraction for South Asian dishes while maintaining low computational overhead.

2.4.3 Hybrid Architectures for Global Feature Modeling

While CNNs excel at local feature extraction, they may struggle with capturing global context. Nfor et al. [nfor2025] propose a hybrid CNN–Vision Transformer model, demonstrating improved performance in scenarios requiring holistic understanding of food presentation.

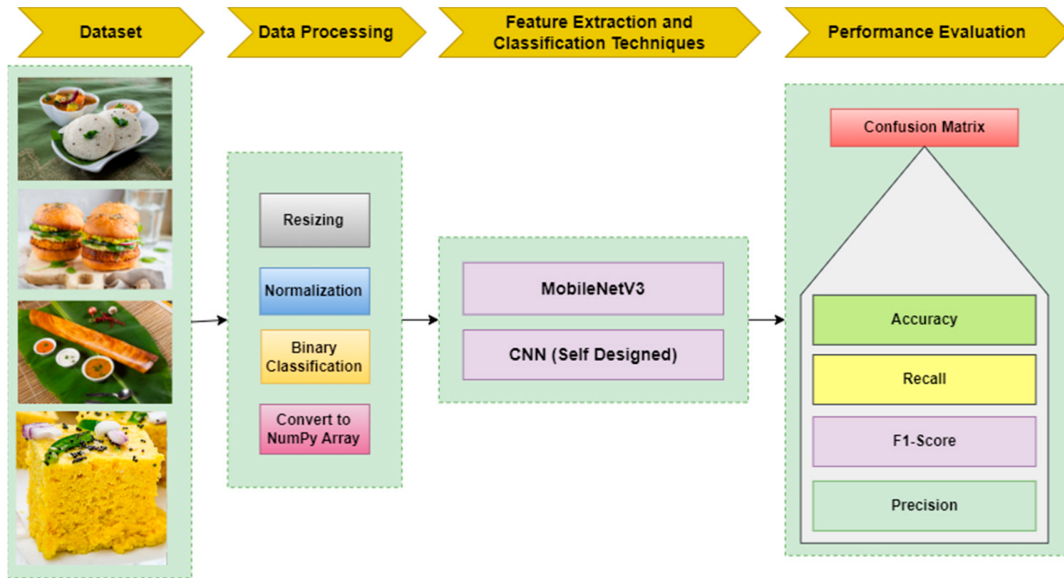


Figure 2.2: An overview design of the classification

2.4.4 From Recognition to Nutritional Estimation

Beyond classification, accurate nutritional estimation remains a critical challenge. Thames et al. [thames2021], through the **Nutrition5k** dataset, demonstrate that incorporating depth information significantly improves portion size estimation. However, RGB-D approaches are not feasible for standard mobile devices, making 2D image-based recognition combined with verified nutritional databases a more practical solution.

2.4.5 Summary of Findings

Paper	Architecture	Dataset	Key Result
Thames et al. [thames2021]	InceptionV2 (RGB-D)	Nutrition5k (5k plates)	Mass MAE reduced from 29.5% to 13.7%
Patel & Modi [patel2023]	MobileNetV3	Indian Food (2.7k images)	95.30% accuracy with augmentation
Philip [philip2024]	EfficientNetB0	Food-101 (101k images)	97.54% training accuracy
Gull et al. [gull2024]	Custom MobileNet	South Asian Subset (6.3k)	98.08% accuracy on regional dishes
Nfor et al. [nfor2025]	CNN + ViT Encoder	Combined (1.55M images)	91.3% accuracy; improved generalization

Table 2.2: Comparison of key literature for mobile food recognition.

2.5 Synthesis and Gap Analysis

Across application-level systems, conversational AI, and visual recognition models, several consistent limitations emerge that hinder the effectiveness of current solutions.

2.5.1 Application Gap: Lack of Regional Adaptation

Existing applications fail to reliably support region-specific cuisines. As noted in the Lancet’s analysis of metabolic risks in Pakistan [Lancet2023], dietary habits are highly localized, yet most platforms treat complex Pakistani dishes as generic entries.

2.5.2 Data Gap: Bias in Visual Recognition

Widely used datasets such as Food-101 [bossard2014] are biased toward Western food categories. While Indian datasets provide partial overlap, they lack representation of Pakistani cuisine, limiting model performance in real-world deployment.

2.5.3 Context Gap: Reliability of AI Systems

LLMs, while powerful, are prone to hallucination when used without grounding. As emphasized by Belkhouribchia and Pen [belkhouribchia2025], there is a lack of systems that combine conversational AI with verified nutritional databases, particularly in the context of Pakistani foods.

2.6 Motivation for SehatGuru

These gaps collectively motivate the development of **SehatGuru**. By integrating a lightweight, fine-grained food recognition model with a Retrieval-Augmented Generation framework grounded in the Pakistan Food Composition Table [PFCT2021], the system aims to deliver an automated, culturally relevant, and reliable nutrition tracking experience tailored to the Pakistani population.

3. Software Requirement Specification (SRS)

This chapter provides detailed specifications of the system under development.

3.1 Functional Requirements

3.1.1 User Authentication and Profile Management

- The system shall allow users to sign up by entering their full name, email address and password.
- The system shall allow users to sign in using their registered credentials.
- The system shall use **Firebase Authentication** for secure login and signup processes.
- The system shall allow users to verify their email address after registration via a verification email.
- The system shall provide a mechanism for users who have forgotten their password to reset it via their email address.
- The system shall send a password reset email containing a unique, time-limited link/token.
- The system shall allow users to set a new password after following the reset link and confirming the new password.
- The system shall display appropriate success and error messages (e.g., “Password has been updated.”, “Email address not found.”, “Reset link is invalid or has expired.”).

- The system shall allow users to update their profile information including weight, height, activity level, fitness goals, target calories, and dietary preferences.
- The system shall store each user's current and target calorie goals and related profile fields.
- The system shall provide an option for users to delete their account and associated personal data permanently.

3.1.2 User Profile and Health Goal Setup

- The system shall guide users through an initial setup flow upon first login to capture demographics and goals.
- The system shall collect user-provided inputs: age, weight (kg), height (cm), sex, activity level (Sedentary/Light/Moderate/Active), and health goal (Lose/Gain/Maintain).
- The system shall collect user preferences including dietary restrictions, disliked foods, budget level, and typical meal times.
- The system shall calculate system-internal daily calorie and macronutrient targets (BMR/TDEE-based) from the provided profile and store them.
- Post-condition: upon completion of setup the user shall be directed to the main app dashboard.

3.1.3 Nutritional Database

- The system shall provide a searchable database containing at least 200 Pakistani dishes with predefined nutritional information per standard unit (per 100 g).
- The system shall store for each food item, at minimum: Calories, Carbohydrates (g), Protein (g), and Fat (g).
- The system shall store portion presets for each dish (e.g., 1 Standard Bowl, 1 Cup, 1 Plate) with associated gram weights.
- The system shall mark a **Top 20 Pakistani Dishes** subset to be used by the camera identification feature.
- The system shall allow administrators to update and maintain the nutritional database entries.

3.1.4 Food Logging (Camera Identification)

- The system shall allow users to capture a new photo or upload an existing image to identify a food item.
- The system shall process uploaded/captured images via a server-level food classification model.
- The classification model shall attempt to identify a single dish from the Top 20 subset and return the best single match when confidence is high.
- The system shall display clear error messages for cases such as dish not recognized.
- The system shall require the user to confirm or correct the identified dish before logging.
- Post-condition: on successful identification and confirmation, the user shall be navigated to the Portion Selection screen.

3.1.5 Food Logging (Manual Search)

- The system shall allow users to search the full dish database by text query (e.g., “chicken karahi”).
- The system shall return a list of matching food items or display a “No results found” message when none match.
- Post-condition: on selection, the user shall be directed to the Portion Selection screen.

3.1.6 Portion Size Selection

- The system shall require users to specify the quantity consumed before logging a selected food item.
- The system shall accept preset quantity selections (e.g., 1, 1.5, 2) combined with portion type (Standard Bowl, Cup, Plate), or accept a direct gram input (e.g., 250 g).
- The system shall compute the nutritional values for the selected portion and add the logged item to the user’s daily log upon confirmation.

3.1.7 Nutritional Calculation

- The system shall calculate estimated calories and macronutrients for a logged item based on the food database entry and the user-specified portion or grams.
- The system shall display in the user’s log: estimated total calories, carbohydrates (g), protein (g), and fat (g).
- The system shall include the calculated values in daily/weekly totals and analytics.

3.1.8 Food Recognition and Classification Implementation Details

- The system shall run image classification at the server level to balance inference speed and client performance.
- The classification model family may include Convolutional Neural Networks (CNNs) and transformer-based image models; the final selection shall balance accuracy, latency, and compute cost.
- The system shall provide a fallback UI that allows manual selection when the classifier is uncertain or the dish is not in the Top 20.
- The system shall use a configurable confidence threshold to decide whether to auto-suggest a single dish or require manual selection.
- The system shall present top-2 suggestions when confidence is low and require user confirmation before logging.

3.1.9 AI Chatbot and Nutritional Assistance

- The system shall integrate a conversational chatbot (“SehatGuru”) powered by the **Gemini API** and a Retrieval-Augmented Generation (RAG) database containing dish metadata and user-specific records.
- The system shall provide a text-based chat interface enabling users to ask nutrition-related queries and receive LLM-generated responses.
- The chatbot shall use the user’s health profile, persisted preferences, and historical meal logs to produce context-aware answers.
- The system shall log chat interactions and user feedback for personalization and model refinement.

- The chatbot shall generate culturally-aware advice prioritizing traditional Pakistani meals and ingredients by looking at the dishes dataset of 200 Pakistani dishes database.
- The chatbot shall allow users to provide feedback on individual responses so it can remember his preferences.

3.1.10 Meal Planning and Meal Prep

- The system shall generate personalized daily and weekly meal plans via the chatbot, aligned to the user's calorie targets and dietary preferences, by selecting from the predetermined database of 200 Pakistani dishes to provide an optimal balance of macro- and micronutrients for the user's goals.
- The system shall allow users to preview, accept, modify, and add AI-generated meal plans directly into their food log or calendar.
- The system shall display aggregated calories and macronutrients for each planned meal and for the entire day/week.
- The system shall allow the user to replace or modify any dish in a generated plan; upon such changes, the chatbot shall automatically re-evaluate and re-generate the affected meal plan to maintain nutritional balance and adherence to the user's targets.

3.1.11 AI Memory and Personalization

- The system shall persist user-specific preferences stated in profile or during chat interactions (e.g., dislikes, allergies, preferred meal times) and make them available to the chatbot
- The chatbot shall adapt suggestions over time using persisted preferences and historic logs to refine future recommendations.
- The system shall periodically analyze recent logs (e.g., last 7 days) against goals to produce proactive, progress-based feedback.
- The system shall incorporate explicit user feedback on chatbot responses into personalization logic.

3.1.12 Progress Dashboard and Analytics

- The system shall provide a centralized dashboard displaying real-time nutritional intake against daily goals (calories and macronutrients).
- The dashboard shall list logged items organized by meal (Breakfast, Lunch, Dinner, Snacks) and show totals (e.g., “1500 / 2200 kcal”).
- The system shall provide historical progress reports and charts (weekly/monthly) for calorie trends, macronutrient averages, and weight history.
- The system shall estimate time-to-goal for weight targets based on logged calorie trends and user progress.
- The system shall allow users to view summaries such as “You hit your protein goal X times this week”.

3.1.13 Personalized Notifications

- The system shall send push/in-app notifications for logging reminders when a user misses typical meal logging windows.
- The system shall send motivational messages based on progress (e.g., streaks, milestones).
- The system shall send inactivity alerts when the user has not used the app for a configurable period (e.g., 3 days).
- The system shall notify users when AI-generated meal plans or meal-prep schedules are available or updated.

3.1.14 Security and Data Management

- The system shall store user data securely in Firebase Firestore and Firebase Cloud Storage with rules limiting access to authenticated users.
- The system shall encrypt sensitive data at rest and in transit and follow best practices for authentication token handling.
- The system shall require user consent for data used in personalization and analytics and follow applicable data privacy best practices.
- The system shall allow users to request deletion of their account and associated data.

- Post-condition: after deletion, the user’s identifiable data shall be removed from Firestore/Storage within a defined period (e.g., 30 days) unless legally required to retain.

3.1.15 User Feedback and Reviews

- The system shall provide an in-app feedback form allowing users to submit reviews about the application.
- The feedback form shall allow users to enter at minimum: a rating (e.g., 1–5 stars), a short title/subject, and written comments.
- The system shall validate feedback submissions (e.g., rating required, comment length limits) and display clear success/error messages.
- The system shall store submitted feedback securely in the backend database (e.g., Firebase Firestore).
- Post-condition: upon successful submission, the system shall acknowledge receipt (e.g., “Thank you for your feedback.”) and save the feedback record.

3.2 Non-functional Requirements

3.2.1 Performance

- The system shall return a food identification result (or a clear failure message) within 5 seconds for at least 90% of requests under normal network conditions.
- The system shall return AI chatbot responses within 8 seconds for at least 90% of typical user queries under normal network conditions.
- The system shall enforce request timeouts and display user-friendly timeout messages when external services (e.g., Gemini API) are slow or unavailable.

3.2.2 Security

- The system shall enforce secure password requirements (minimum 6 characters) during account registration.
- The system shall require email verification before granting full access to protected features.

- The system shall use token-based authentication (JWT access/refresh tokens) with expiration for protected backend endpoints.
- All data transmitted between the mobile application and backend servers shall be encrypted using HTTPS (TLS 1.2 or higher) in production deployments.

3.2.3 Reliability

- The system shall maintain 90% uptime during the MVP testing period to ensure reliable meal tracking.
- If the image classification model fails or cannot identify a dish, the system shall automatically provide the option to redirect users to manual food search without application crash or data loss to ensure meal logging continuity.
- The system shall display user-friendly, contextual error messages for all failure scenarios including network connectivity issues, authentication errors, image processing failures, and invalid data entries to guide users on appropriate next steps.
- If Firebase services are unavailable/misconfigured, the system shall degrade gracefully by disabling Firebase-dependent features while keeping non-Firebase features (e.g., food detection testing) operational when possible.

3.2.4 Usability

- New users shall be able to complete account registration, health profile setup, and log their first meal within 10 minutes without external assistance to ensure a smooth onboarding experience.
- The application shall provide a complete English language interface for all user-facing features, including menus, labels, error messages, and chatbot interactions.
- The application shall target Android devices running Android 8.0 (API level 26, Oreo) or higher versions.
- The application shall target iOS devices running iOS 16.0 or higher versions.
- The application shall follow standard mobile UI/UX patterns and design guidelines commonly used in popular Pakistani mobile applications to ensure intuitive navigation and minimal learning curve.

3.2.5 Compliance

- The application shall display a prominent disclaimer during initial setup stating that the nutritional advice and recommendations provided are for general informational and wellness purposes only and are not a substitute for professional medical advice, diagnosis, or treatment.
- The system shall obtain explicit user consent during the registration and onboarding process before collecting any health-related information (weight, height, age, health goals, dietary preferences) to ensure users maintain control of their personal data.
- The system shall clearly inform users when AI features use their profile/log data for personalization.
- The system shall allow users to opt out of AI personalization and delete stored AI memory/preference records.

3.2.6 Data Integrity

- The system shall validate all user health profile inputs to maintain data accuracy, including:
 - Weight: 20 kg – 300 kg
 - Height: 100 cm – 250 cm
 - Age: 13 years – 100 years
 - Portion quantities: Greater than 0

Invalid entries shall be rejected with clear error messages indicating the acceptable range.

- The food classification model shall achieve a minimum accuracy of 75% when identifying dishes from the “Top 20 Pakistani Dishes” subset in testing scenarios with clear, well-lit images.
- The system shall use a unified calculation algorithm for estimating calories and macronutrients regardless of input method, ensuring consistent results for identical food items and portion sizes.
- The system shall validate user inputs on both client-side (for immediate feedback) and server-side (for security and integrity).

3.2.7 Maintainability

- Core system modules (authentication, food logging, image classification integration, nutritional calculation, chatbot integration) shall include inline code comments and README documentation explaining functionality, dependencies, and setup procedures.
- The application shall follow a modular architecture pattern separating major functional components:
 - Authentication Module
 - Food Logging Module
 - AI Chatbot Module
 - Dashboard and Analytics Module
 - Notification Module

Each module shall have clearly defined interfaces to minimize dependencies and enable independent development and testing.

3.2.8 Availability and Rate Limiting

- The system shall implement server-side rate limiting for AI chatbot endpoints to prevent abuse and control operational costs.
- The system shall return appropriate error messages when rate limits are exceeded.

3.2.9 Observability

- The system shall log backend errors and request failures (authentication, chatbot, and food detection) for debugging and monitoring.

3.3 External Interfaces

We expect every project to have at least of the following subsections. This section must be aligned with your project deliverables. Please consult with your project supervisor regarding which of the following section(s) you should include in your report

3.3.1 User Interfaces

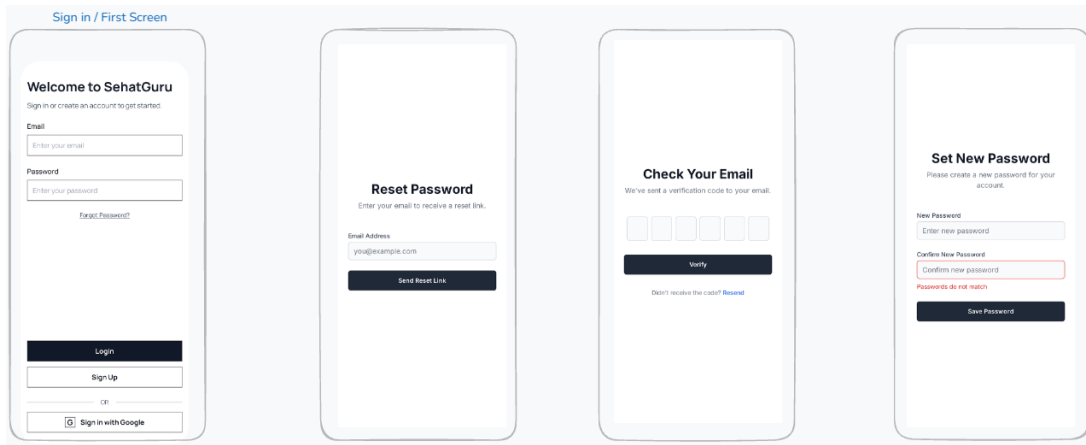


Figure 3.1: Login and Password Recovery Screens

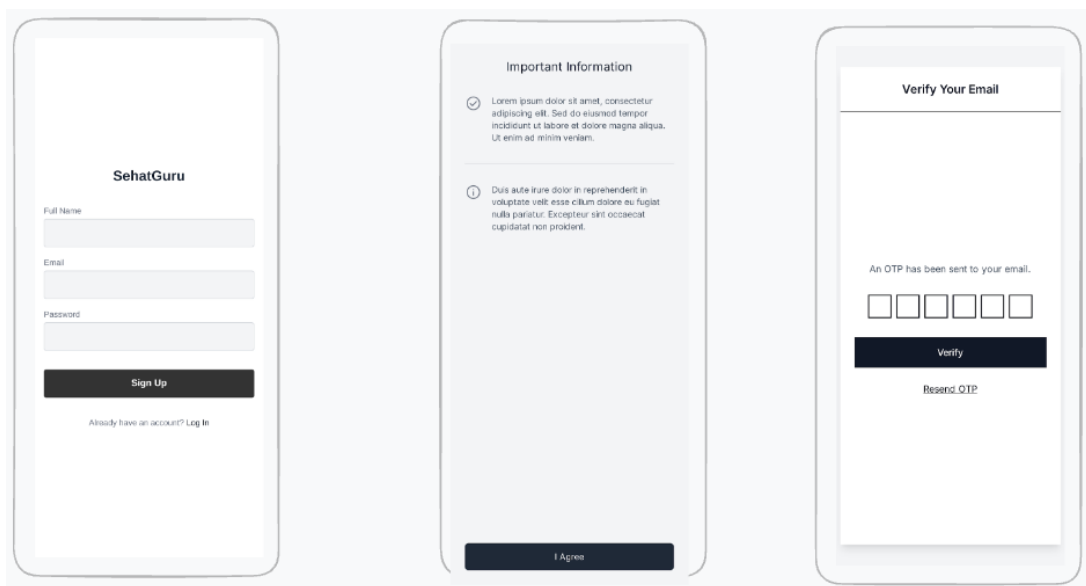


Figure 3.2: Signup, Terms, and Email Verification Screens

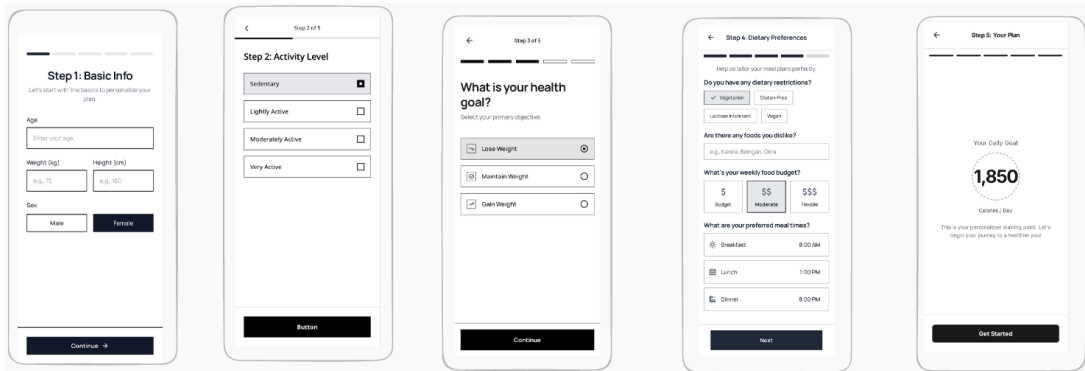


Figure 3.3: User Profile Setup and Goal Selection Screens

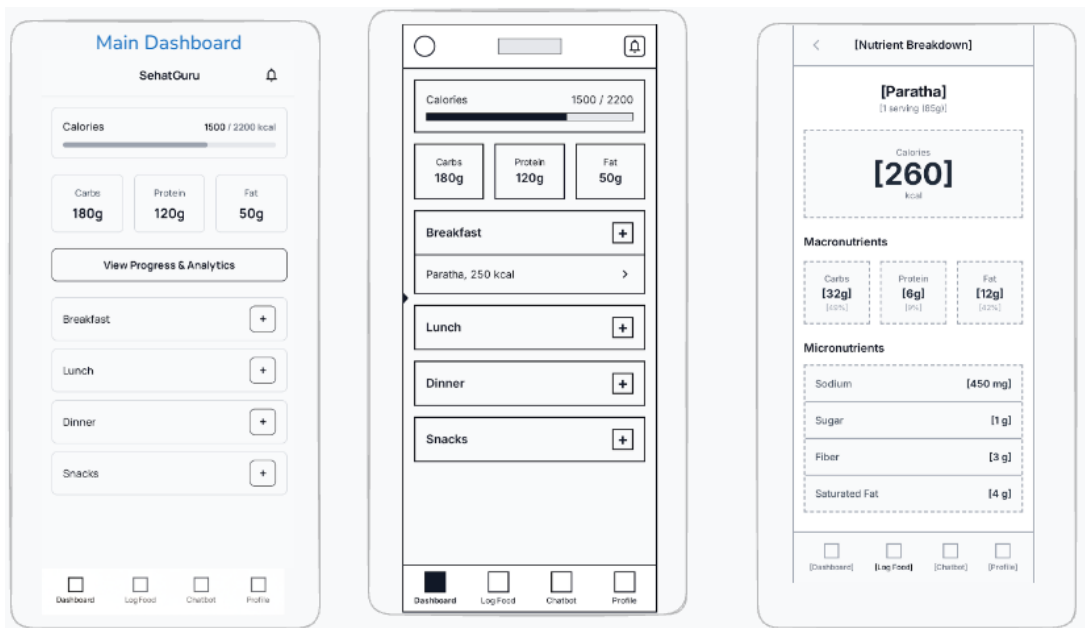


Figure 3.4: Main Dashboard and Nutritional Breakdown Screens

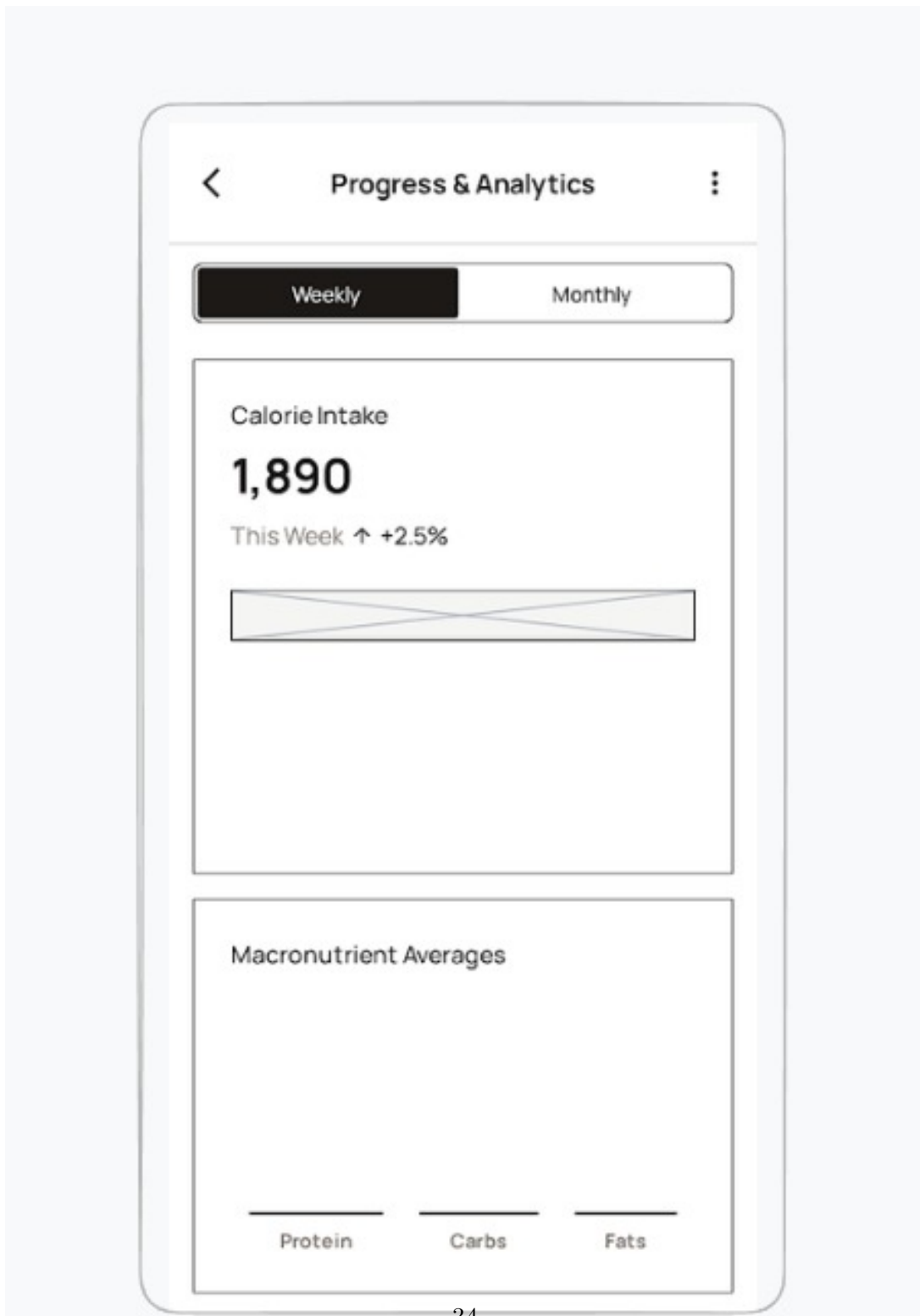


Figure 3.5: Progress and Analytics Overview Screen

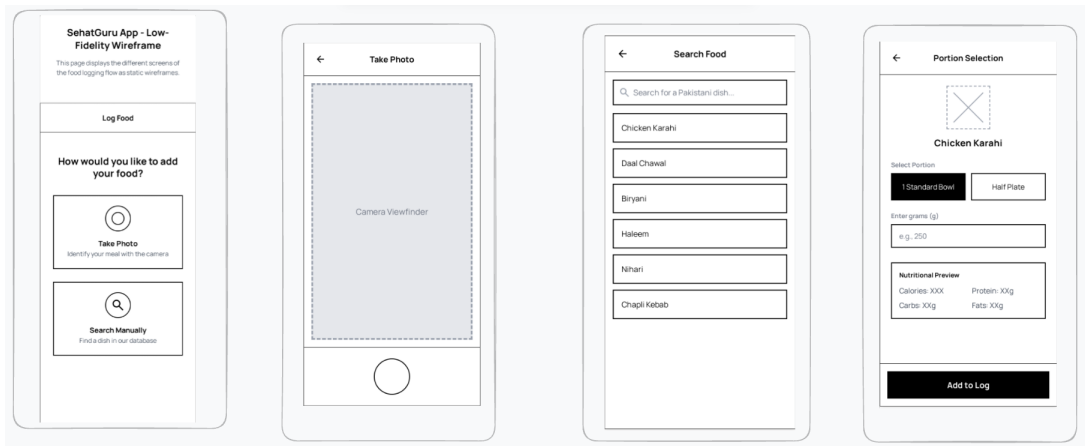


Figure 3.6: Food Logging Flow: Capture, Search, and Portion Selection Screens

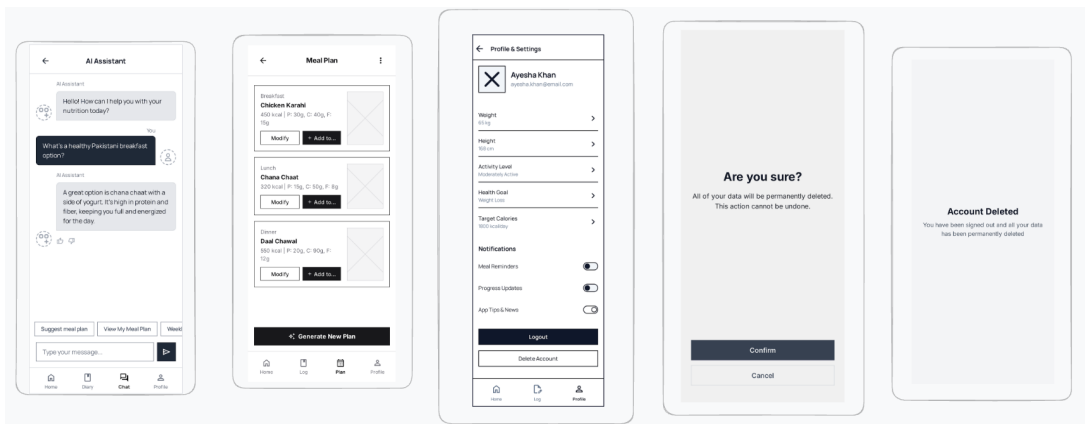


Figure 3.7: Chatbot, Meal Planning, Profile, and Account Management Screens

3.3.2 Application Program Interface (API)

The SehatGuru system utilizes a RESTful API architecture to facilitate communication between the mobile client and the AI-powered backend. The backend is developed using FastAPI, providing high-performance endpoints for the PyTorch-based image recognition model and the RAG-enabled chatbot.

Core interfaces include the Predict API, which accepts image data to return nutritional estimates for Pakistani cuisine, and the Query API, which interfaces with a vector database (via LangChain) to provide culturally-aware health advice. All data is exchanged in JSON format, and requests are secured using JWT for user session management.

3.3.3 Hardware/Communication Interfaces

There are no hardware interfaces required.

3.4 Use Cases

This section presents detailed use cases of our system.

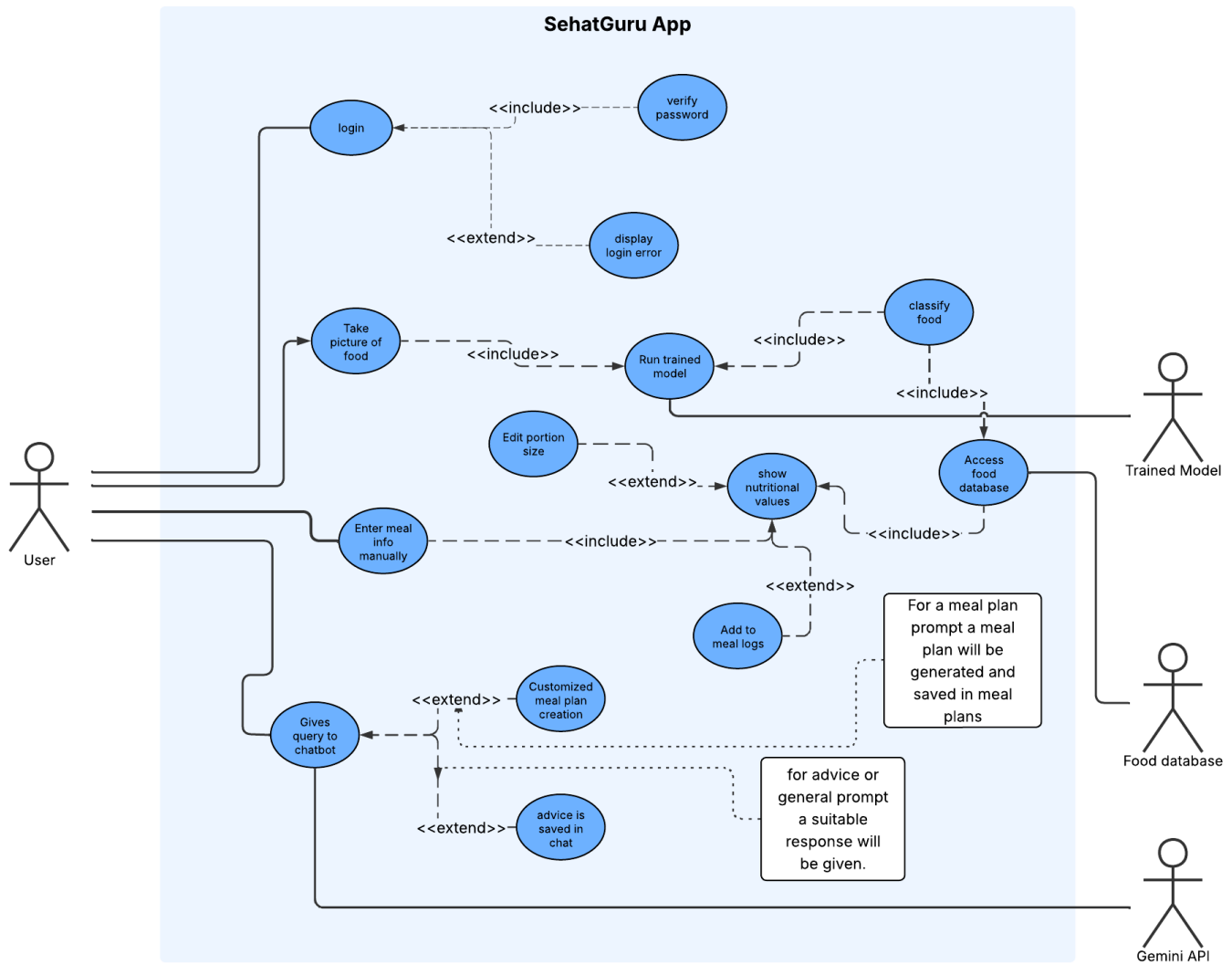


Figure 3.8: Use Case Diagram for SehatGuru

3.5 Datasets

The SehatGuru system relies on three primary datasets:

1. **Pakistani Food Nutritional Database:** A curated database of 200+ local dishes. Data was sourced from the *Pakistan Food Composition Table* and verified by nutritional experts.
2. **Indian Food Nutritional Database:** A curated database of 200+ local dishes. Data was sourced from the *indian Food Composition Table* and verified by nutritional experts.
3. **Image Classification Subset (Top 20):** A curated set of 20 commonly consumed Pakistani dishes used for training the vision model, comprising 5,000+ labeled images capturing diverse lighting conditions and plating styles typical in local households. Notably, this dataset was **collected, curated, and annotated entirely by the SehatGuru team**, ensuring both quality and cultural relevance. Further details are provided in Section 5.

3.6 System Diagram

This diagram gives a high-level view of the different components of our system and the interactions between them. Each component and the particular tools/technologies/libraries used to build it are described.

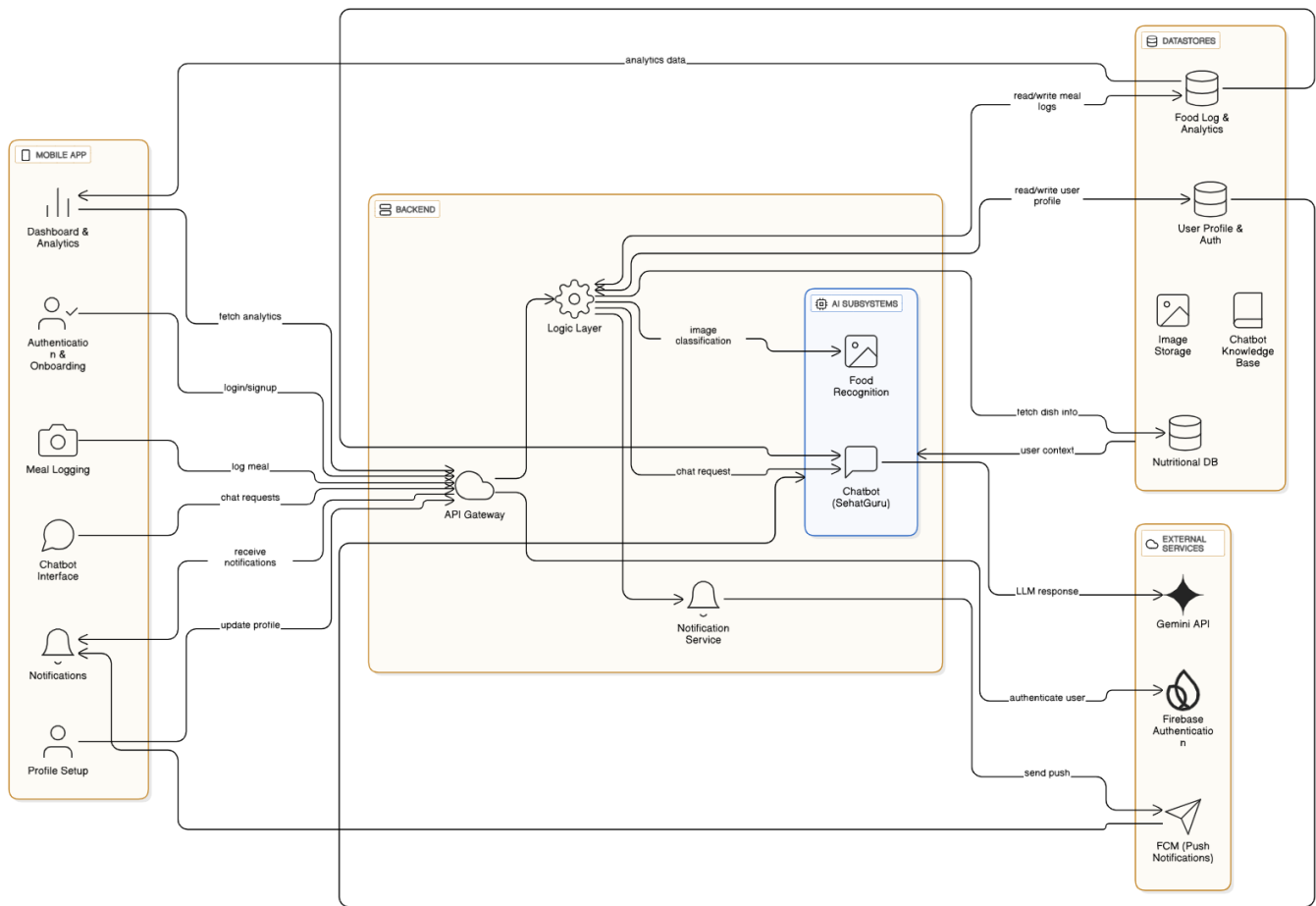


Figure 3.9: System Block Diagram of SehatGuru

4. Software Design Specification (SDS)

This chapter provides important artifacts related to design of our project.

4.1 Software Design

This section details the architectural blueprint and object-oriented design of the Sehat Guru application. It translates the requirements into a technical structure comprising distinct layers and interacting objects.

4.1.1 System Architecture

Sehat Guru follows a **Client-Server Architecture** with an API Gateway pattern. The system is divided into the Mobile Client (Frontend) and the Backend Logic Layer, which orchestrates interactions between AI subsystems, datastores, and external services.

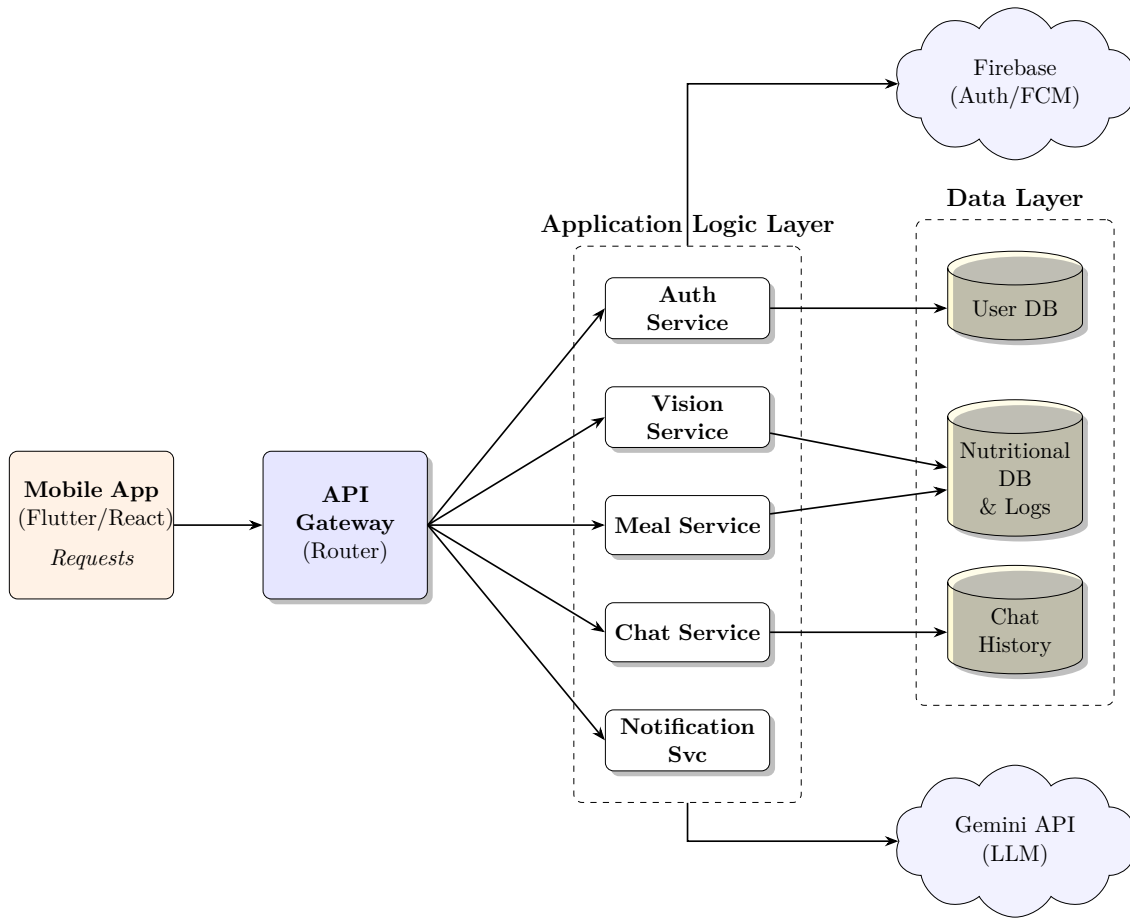


Figure 4.1: System Architecture Diagram

4.1.2 UML Class Diagram

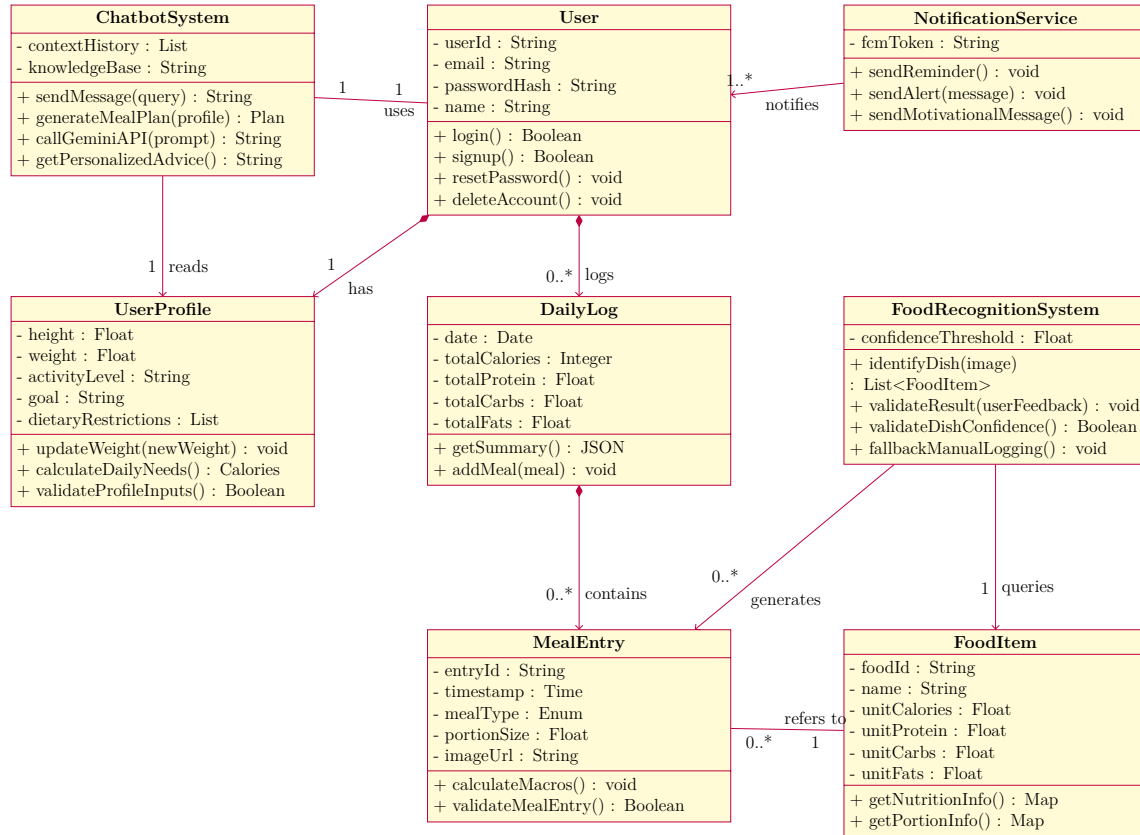


Figure 4.2: UML Class Diagram for Sehat Guru Application

4.1.3 Class Design Details

Class: User The **User** class represents a registered user in the SehatGuru application. It contains personal information and authentication methods.

- **Attributes:**

- **userId:** A unique identifier for the user.
- **email:** The user’s email address.
- **passwordHash:** The hashed password for the user’s account.
- **name:** The full name of the user.

- **Methods:**

- `login()`: Authenticates the user based on email and password.
- `signup()`: Registers a new user in the system.
- `resetPassword()`: Resets the user's password.
- `deleteAccount()`: Deletes the user account and all associated data.

Class: UserProfile The `UserProfile` class manages the user's health profile and goal settings. This includes tracking physical metrics and preferences.

- **Attributes:**

- `height`: The user's height in centimeters.
- `weight`: The user's weight in kilograms.
- `activityLevel`: The user's activity level (e.g., sedentary, active).
- `goal`: The health goal of the user (e.g., lose weight, gain muscle).
- `dietaryRestrictions`: A list of foods the user must avoid due to allergies or dietary preferences.

- **Methods:**

- `updateWeight(newWeight)`: Updates the user's weight in the profile.
- `calculateDailyNeeds()`: Calculates the user's daily caloric needs based on their profile data.
- `validateProfileInputs()`: Ensures that the user's input data is within valid ranges.

Class: DailyLog The `DailyLog` class tracks the user's meal logs for a specific day, aggregating nutritional information.

- **Attributes:**

- `date`: The date of the log entry.
- `totalCalories`: The total number of calories logged for the day.
- `totalProtein`: The total protein intake for the day.
- `totalCarbs`: The total carbohydrate intake for the day.
- `totalFats`: The total fat intake for the day.

- **Methods:**

- `getSummary()`: Returns a summary of the day's meal log in JSON format.
- `addMeal(meal)`: Adds a meal entry to the daily log.

Class: MealEntry The `MealEntry` class represents an individual meal entry in the user's daily log. It includes details such as the meal type and portion size.

- **Attributes:**

- `entryId`: A unique identifier for each meal entry.
- `timestamp`: The time the meal was logged.
- `mealType`: The type of meal (e.g., breakfast, lunch, dinner).
- `portionSize`: The portion size of the meal in grams.
- `imageUrl`: A link to an image of the meal, if available.

- **Methods:**

- `calculateMacros()`: Calculates the macros (calories, protein, carbs, fats) for the meal based on the portion size.
- `validateMealEntry()`: Validates the meal entry to ensure all data is correct.

Class: FoodItem The `FoodItem` class contains nutritional information for a specific food item.

- **Attributes:**

- `foodId`: A unique identifier for each food item.
- `name`: The name of the food item.
- `unitCalories`: The number of calories per unit of food.
- `unitProtein`: The amount of protein per unit of food.
- `unitCarbs`: The amount of carbohydrates per unit of food.
- `unitFats`: The amount of fats per unit of food.

- **Methods:**

- `getNutritionInfo()`: Returns a map of the nutritional information for the food item.
- `getPortionInfo()`: Returns information about the available portion sizes for the food item.

Class: FoodRecognitionSystem The `FoodRecognitionSystem` class is responsible for recognizing food from images and validating results.

- **Attributes:**

- `confidenceThreshold`: A threshold for confidence level in food recognition.

- **Methods:**

- `identifyDish(image)`: Identifies the dish from an image using food recognition.
- `validateResult(userFeedback)`: Validates the identified dish based on user feedback.
- `validateDishConfidence()`: Ensures that the identified dish meets the minimum confidence threshold.
- `fallbackManualLogging()`: Provides an option for manual logging if the food recognition fails.

Class: ChatbotSystem The `ChatbotSystem` class powers the AI chatbot, which offers nutrition advice and meal planning.

- **Attributes:**

- `contextHistory`: A list of past interactions with the user.
- `knowledgeBase`: A repository of nutritional information and common questions.

- **Methods:**

- `sendMessage(query)`: Sends a query to the chatbot and returns the response.
- `generateMealPlan(profile)`: Generates a personalized meal plan based on the user's profile.
- `callGeminiAPI(prompt)`: Calls the Gemini API to generate a response.
- `getPersonalizedAdvice()`: Provides personalized nutritional advice based on user data.

Class: NotificationService The `NotificationService` class is responsible for sending notifications to users, such as reminders or alerts.

- **Attributes:**
 - `fcmToken`: The token for Firebase Cloud Messaging to send notifications to the user.
- **Methods:**
 - `sendReminder()`: Sends a reminder notification to the user.
 - `sendAlert(message)`: Sends an alert message to the user.
 - `sendMotivationalMessage()`: Sends a motivational message to encourage the user.

4.2 Data Design

The data design section describes the database structure for the SehatGuru application, presenting a normalized relational model showing entities, attributes, primary/foreign keys, and relationships with correct cardinalities. This design directly maps to the UML Class Diagram to ensure consistency between the conceptual and physical models.

4.2.1 Overview

Database Technology: Firebase Firestore (NoSQL Document Database)

Normalization Level: Third Normal Form (3NF)

Total Entities: 10 tables

- **Core Tables :** Users, UserProfiles, DailyLogs, MealEntries, FoodItems, ChatMessages, Notifications
- **Supporting Tables:** PortionPresets, MealPlans, ImageStorage

4.2.2 Entity-Relationship Diagram

Figure 4.3 illustrates the complete database schema with all entities, attributes, primary keys (underlined), foreign keys, and relationships with cardinalities.

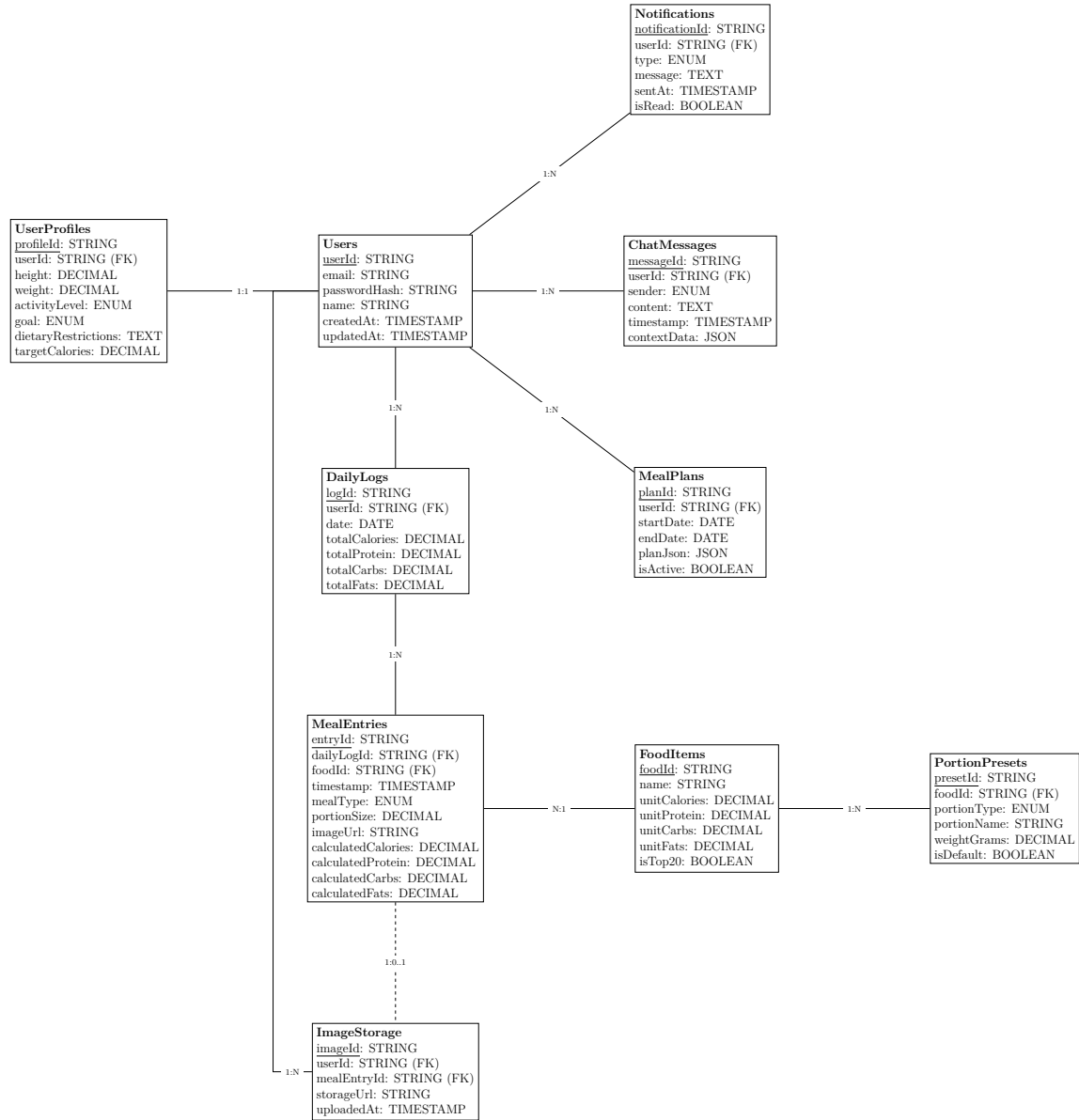


Figure 4.3: Entity-Relationship Diagram for SehatGuru Database

4.2.3 Database Schema (Normalized Relational Model)

The database is designed in Third Normal Form (3NF) to eliminate redundancy while maintaining data integrity. Table 4.1 presents the complete schema with all entities, attributes, data types, and constraints.

Table 4.1: SehatGuru Database Schema

Table Name	Primary Key	Attributes & Foreign Keys
<i>Core Tables</i>		
Users	userId	email (UNIQUE), passwordHash, name, createdAt, updatedAt
UserProfiles	profileId	userId (FK → Users, UNIQUE), height, weight, activityLevel, goal, dietaryRestrictions, targetCalories
DailyLogs	logId	userId (FK → Users), date, totalCalories, totalProtein, totalCarbs, totalFats
MealEntries	entryId	dailyLogId (FK → DailyLogs), foodId (FK → FoodItems), timestamp, mealType, portionSize, imageUrl, calculatedCalories, calculatedProtein, calculatedCarbs, calculatedFats
FoodItems	foodId	name (UNIQUE), unitCalories, unitProtein, unitCarbs, unitFats, isTop20
ChatMessages	messageId	userId (FK → Users), sender, content, timestamp, contextData
Notifications	notificationId	userId (FK → Users), type, message, sentAt, isRead
<i>Supporting Tables</i>		
PortionPresets	presetId	foodId (FK → FoodItems), portionType, portionName, weightGrams, isDefault
MealPlans	planId	userId (FK → Users), startDate, endDate, planJson, isActive
ImageStorage	imageId	userId (FK → Users), mealEntryId (FK → MealEntries), storageUrl, uploadedAt

4.2.4 UML-to-Database Mapping

Table 4.2 demonstrates how each UML class maps to database entities, ensuring consistency between conceptual and physical models.

Table 4.2: UML Class to Database Table Mapping

UML Class	Database Table	Key Attributes & Methods Supported
User	Users	userId, email, passwordHash → login(), signup(), resetPassword(), deleteAccount()
UserProfile	UserProfiles	height, weight, goal → calculateDailyNeeds(), updateWeight()
DailyLog	DailyLogs	date, total* fields → getSummary(), addMeal()
MealEntry	MealEntries	foodId, portionSize, calculated* fields → calculateMacros()
FoodItem	FoodItems	unit* nutritional values → getNutritionInfo(), getPortionInfo()
FoodRecognitionSystem	(Service)	Queries FoodItems (isTop20=true); no persistent storage
ChatbotSystem	ChatMessages	contextHistory → sendMessage(), callGeminiAPI()
ChatbotSystem	MealPlans	generateMealPlan() → stores planJson
NotificationService	Notifications	fcmToken, type, message → sendReminder(), sendAlert()

4.2.5 Entity Relationships

Table 4.3 summarizes all relationships with cardinalities matching the UML diagram.

Table 4.3: Entity Relationships Summary

Parent Entity	Child Entity	Cardinality	Relationship Type
Users	UserProfiles	1:1	Composition (CASCADE)
Users	DailyLogs	1:N	Composition (CASCADE)
DailyLogs	MealEntries	1:N	Composition (CASCADE)
MealEntries	FoodItems	N:1	Association (RESTRICT)
Users	ChatMessages	1:N	Association (CASCADE)
Users	Notifications	1:N	Association (CASCADE)
Users	MealPlans	1:N	Association (CASCADE)
FoodItems	PortionPresets	1:N	Composition (CASCADE)
MealEntries	ImageStorage	1:0..1	Optional Association (SET NULL)

4.2.6 Design Justification

Consistency with UML

This database design maintains complete alignment with the UML class diagram:

- **Direct Mapping:** Each UML class has a corresponding database table with matching attributes
- **Preserved Relationships:** All UML compositions (filled diamonds) and associations (solid lines) are implemented as foreign key constraints with correct cardinalities
- **Service Classes:** FoodRecognitionSystem is correctly identified as a stateless service that queries FoodItems but stores no data

Normalization (3NF)

The schema achieves Third Normal Form:

- **1NF:** All attributes contain atomic values
- **2NF:** No partial dependencies; all non-key attributes depend on the entire primary key
- **3NF:** No transitive dependencies

Strategic Denormalization: MealEntries.calculated* and DailyLogs.total* fields store pre-computed values to optimize query performance (10x faster dashboard)

loads) with minimal storage overhead.

SRS Coverage

All functional requirements are supported:

- **Authentication (SRS 3.1):** Users table
- **Profile Setup (SRS 3.2):** UserProfiles table
- **Food Database (SRS 3.3):** FoodItems table (~200 dishes)
- **Meal Logging (SRS 3.4-3.5):** MealEntries, ImageStorage
- **Portion Selection (SRS 3.6):** PortionPresets table
- **AI Chatbot (SRS 3.9):** ChatMessages table
- **Meal Planning (SRS 3.10):** MealPlans table
- **Dashboard (SRS 3.12):** DailyLogs aggregation
- **Notifications (SRS 3.13):** Notifications table

4.3 Technical Details

Our project does not have persistent data so we have no ERD. Instead we explain here the technical details of the algorithm we use. These include the inputs and the outputs, how and where these algorithms fit in our tool chain, the techniques used in these algorithms, etc.

5. Dataset

5.1 Dataset

The dataset curation for this project was divided into three distinct phases: the identification of high-frequency regional dishes, the collection and cleaning of an image corpus for model training, and the synthesis of a comprehensive nutritional database.

5.1.1 Selection of Top 20 Pakistani Dishes

To ensure the application caters to the most common dietary habits in Pakistan, a frequency analysis of popular dishes was conducted. A custom scraping script was developed to crawl over 300 online culinary articles and regional food rankings. Using a Large Language Model (LLM) for entity extraction, we identified the dishes most frequently mentioned across these sources.

After a data cleaning process that involved removing approximately 100 redundant or irrelevant articles, the scope was narrowed to the top 20 dishes based on their frequency of appearance. The full list of these identified dishes, along with their observed frequencies from the scraped data, is detailed in Table 5.1. While there is an intention to expand this dataset in the future through real-time user analytics within the application, the current scope is limited to these 20 items due to the project’s developmental timeframe.

Table 5.1: Frequency Distribution of Top 20 Identified Dishes

Rank	Dish	Frequency	Rank	Dish	Frequency
1	Biryani	173	11	Sajji	17
2	Karahi	124	12	Samosas	16
3	Pulao	113	13	Pakoras	16
4	Nihari	49	14	Dahi Baray / Bhalla	16
5	Haleem	46	15	Seekh Kabab	13
6	Saag Dishes	32	16	Jalebi	13
7	Kheer	25	17	Gajar ka Halwa	12
8	Gulab Jamun	25	18	Kulfi	12
9	Chaat	19	19	Paratha	12
10	Chapli Kebab	17	20	Zarda	12

5.1.2 Image Dataset Curation

Following the identification of the target dishes, a secondary scraper was utilized to aggregate images from various search engines. However, raw web-scraped data presented several challenges, including “noisy” images featuring chefs, packaged spice mixes, or unrelated culinary items.

To ensure high-quality training data for the classification model, a manual cleaning phase was performed. This involved filtering out unrelated images and cropping relevant samples to emphasize the dish itself. The resulting primary training dataset consists of over 5,000 images, with a distribution ranging between 150 to 320 images per dish to account for the varying availability of high-quality regional samples. As illustrated in Figure 5.1, classes such as *Gulaab Jamun* and *Kulfi* show the highest density, while the dataset remains robust across all 20 categories. Figure 5.2 showcases the diversity and quality of the visual data after the cleaning and cropping process.

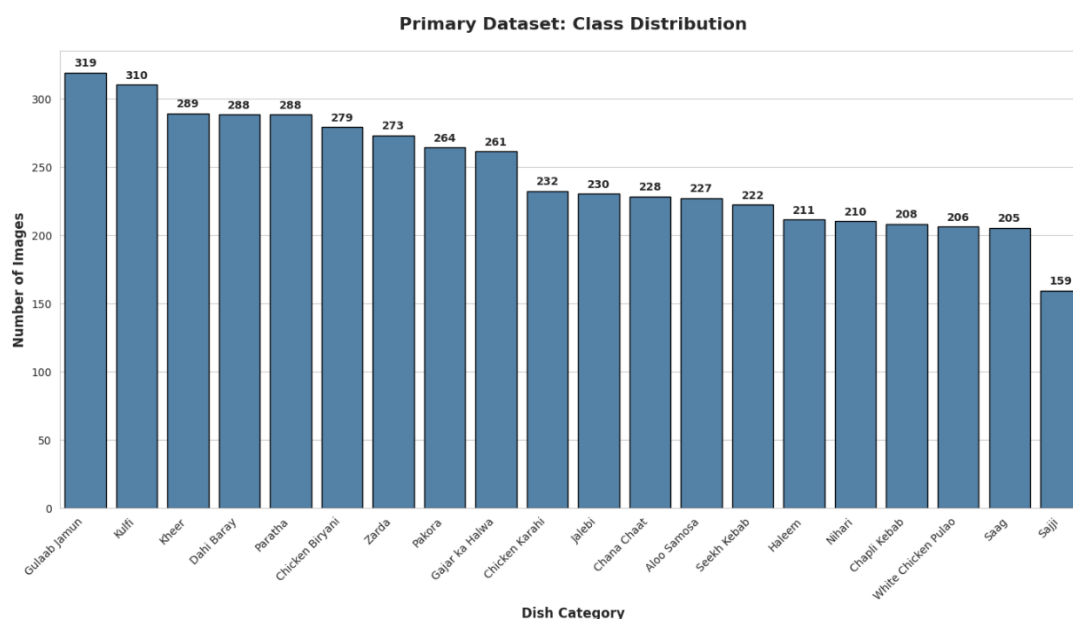


Figure 5.1: Class distribution of the primary training dataset (5,000+ images).

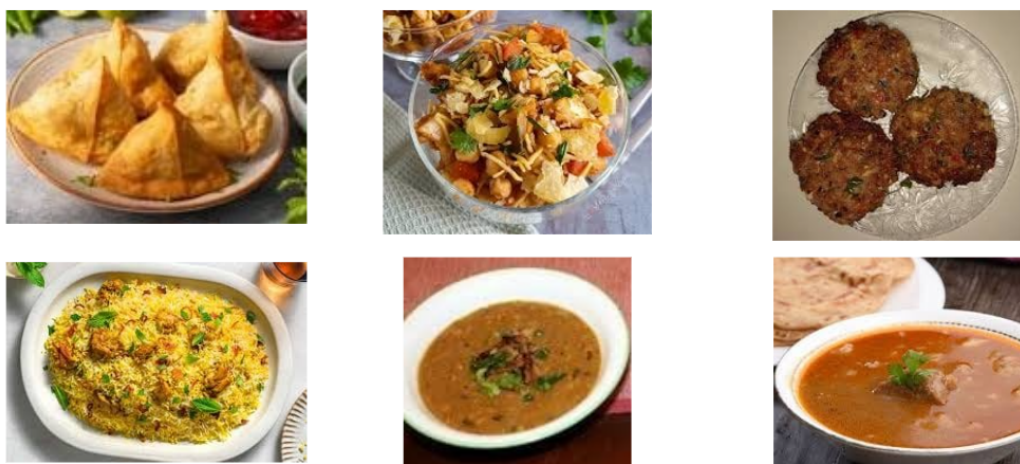


Figure 5.2: Visual samples of cleaned and cropped food images used for model training.

Furthermore, a specialized validation dataset was manually curated to bridge

the gap between web-sourced data and real-world application usage. While Google Images often provide samples in idealized or studio lighting, this secondary dataset consists of approximately 200 images captured in natural environments. As shown in Figure 5.3, this dataset maintains a consistent distribution of roughly 10 images per class, ensuring a balanced evaluation across the entire menu.

These images were sourced through original photography and social media platforms like Instagram to capture variations in lighting, plating styles, and camera angles (see Figure 5.4). This dataset served as the final benchmark for validating the model’s performance in real-world scenarios.

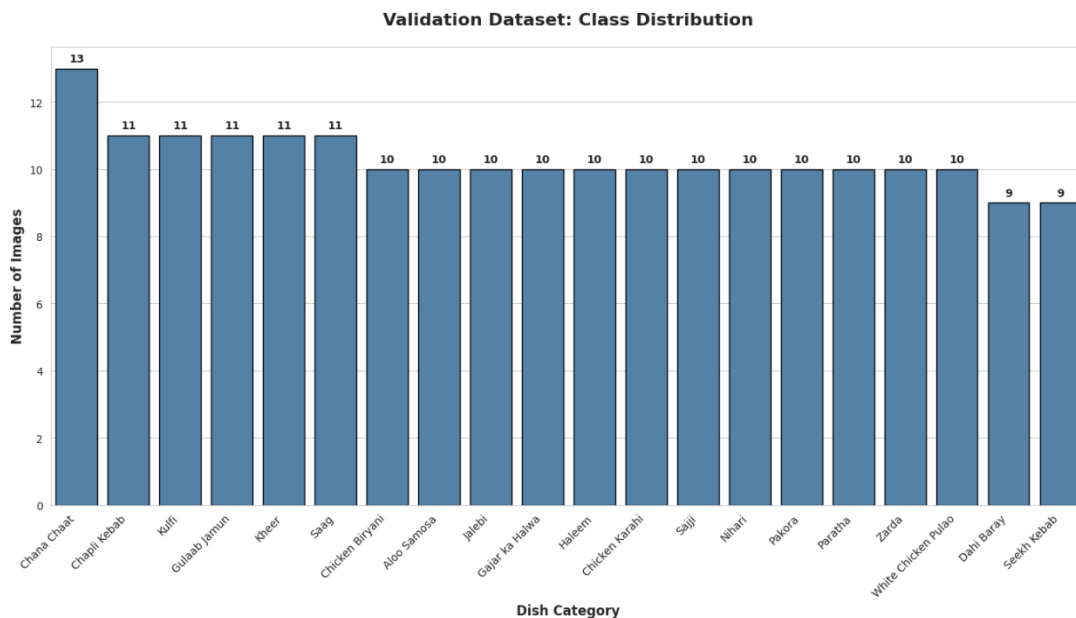


Figure 5.3: Class distribution of the manually curated real-world validation dataset.



Figure 5.4: Sample images from the validation set illustrating real-world lighting and diverse plating.

5.1.3 Nutritional Database Synthesis

The final component of the data pipeline was the creation of a comprehensive nutritional database. Data was synthesized from two primary sources: official Pakistani and Indian food authority documentation.

While the Pakistani documentation provided essential macronutrients and selected micronutrients, it was found to be incomplete for several complex dishes. To resolve this, we consulted with Dr. Humaira Jamshed, who recommended utilizing the more extensive Indian dataset for shared regional dishes.

The resulting database contains approximately 1,020 entries. The categorical distribution and specific sources for the various food items within our final dataset are visualized in Figure 5.5. As illustrated in the figure:

- **Green:** Entries sourced exclusively from Pakistani food documentation.
- **Red:** Regional dishes common to both countries, where the more complete Indian dataset was preferred.

- **Blue:** Dishes from our top 20 list not found in Pakistani records, supplemented entirely by the Indian dataset.

food_name	energy_kcal	carb_g	protein_g	fat_g	free_sugar_g	fibre_g	calcium_mg	iron_mg	energy_kj	sfa_mg	mufa_mg	pufa_mg	cholesterol_mg	phosphorus_mg	magnesium_mg	sodium_mg
Kulcha	156	23.20	10.20	2.00	0.00	1.00	31.00	1.20	652.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Kulcha (Meat balls)	152	14.50	13.50	4.00	0.00	0.00	1.10	1.00	636.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Chapati Kebab	134	12.80	5.00	6.60	0.00	1.50	0.30	1.50	560.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Suji (Roasted wheat)	157	1.50	17.20	6.60	0.00	0.00	15.00	1.80	656.40	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Chicken Biryani	197	23.80	9.10	6.70	0.00	0.80	52.00	0.90	825.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Zarda (Sweet Rice)	246	46.50	4.50	4.00	0.00	0.50	0.40	0.50	1029.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Balti Mutton Curry	96.00	12.60	5.20	2.50	0.00	1.00	0.80	1.10	401.30	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Chapati/Roti	282	54.70	9.70	1.60	1.00	2.50	31.00	3.30	844.75	1737.22	796.67	538.89	6.83	175.00	69.44	58.61
Kidney bean curry (Rajmah curry)	162	24.30	11.70	1.50	2.49	2.50	1.00	0.80	677.60	724.31	1085.92	3529.48	0.00	123.00	58.61	58.61
Plain pulao	179	10.90	11.00	6.00	1.15	1.10	1.00	0.30	749.20	585.27	946.96	2865.54	0.00	42.93	12.30	12.30
Shanmi kebab	137	16.30	10.20	3.00	0.31	1.05	1.00	0.90	573.40	9453.59	15312.34	44621.43	24.60	53.08	11.03	11.03
Chicken curry	167	11.70	9.80	8.50	1.79	1.41	3.00	0.50	688.00	821.08	1119.97	2095.92	31.25	107.61	21.89	21.89
Fried fish (Indian style) (Tali hui machli)	237	1.80	22.70	14.20	0.40	0.30	42.00	1.50	991.90	6335.42	14107.74	43531.64	6.57	46.90	10.23	10.23
Semolina halwa (Suji ka halwa)	337	52.10	2.90	12.00	13.96	12.00	0.70	0.10	1409.90	7412.72	3906.35	857.21	30.82	27.64	13.88	13.88
Rice kheer (Chawal ki kheer)	125	18.20	3.60	2.70	8.02	0.20	137.00	0.20	522.50	1764.00	866.18	129.29	0.00	65.41	7.33	7.33
Carrot halwa (Gajar ka halwa)	410	42.00	5.10	24.20	16.85	2.10	150.00	1.10	1713.00	5410.09	2812.91	694.99	14.66	98.45	22.96	22.96
Hot tea (Garam Chai)	16.14	2.58	0.39	9.53	2.58	0.00	14.20	0.02	68.16	321.56	144.18	15.39	0.00	11.99	1.94	1.94
Instant coffee	23.16	3.65	0.64	0.75	3.62	0.00	20.87	0.06	97.77	451.17	202.33	23.00	0.00	18.19	3.63	3.63
Espresso coffee	51.54	6.62	1.75	2.14	6.53	0.00	58.10	0.15	217.00	1291.67	580.16	66.21	0.00	50.25	8.57	8.57
Iced tea	10.34	2.70	0.03	0.01	2.70	0.00	1.18	0.02	44.09	5.47	0.00	5.47	0.00	0.58	0.39	0.39
Raw mango drink (Aam panna)	35.92	9.05	0.16	0.03	7.49	0.61	7.08	0.14	152.39	6.14	5.89	9.43	0.00	3.34	4.25	4.25
Fruit Punch (with fresh juices)	36.12	9.38	0.14	0.03	9.25	0.06	5.07	0.10	153.89	6.21	1.30	10.08	0.00	10.24	3.20	3.20
Fruit Punch (with squashes)	23.13	5.99	0.07	0.02	5.88	0.17	3.30	0.06	98.22	5.23	1.63	7.07	0.00	9.27	1.88	1.88
Lemonade	20.80	5.48	0.03	0.01	5.47	0.02	1.84	0.05	88.66	4.65	0.64	5.30	0.00	0.67	0.73	0.73
Lem-o-gin	21.52	5.55	0.08	0.03	5.38	0.13	2.20	0.09	91.63	9.41	1.47	13.20	0.00	1.59	1.86	1.86
Cumin infused water (Jeera/Zaara ka pani)	9.09	1.86	0.17	0.11	1.46	0.46	10.84	0.32	38.23	14.50	40.04	33.53	0.00	4.47	5.28	5.28
Coco pine cooler	32.75	5.72	0.58	1.00	5.52	0.26	17.81	0.13	138.47	627.31	248.96	41.50	3.40	13.47	6.59	6.59
Summer cooler	22.43	5.43	0.37	0.64	5.21	0.20	8.13	0.25	95.22	10.71	1.87	16.85	0.00	11.46	7.62	7.62
Hot cocoa	90.10	9.23	3.36	4.56	9.01	0.00	113.66	0.31	378.24	2752.15	1257.17	139.48	0.00	101.41	16.05	16.05
Cold coffee with ice cream	67.85	11.24	1.57	2.11	11.10	0.01	53.29	0.12	286.61	1283.98	567.96	67.66	1.17	44.98	7.26	7.26
Banana milkshake (Kela milkshake)	65.31	9.15	1.84	2.37	7.95	0.26	82.64	0.13	274.33	1423.07	634.17	87.19	0.00	52.77	7.52	7.52
Mango milkshake (Aam milkshake)	56.90	7.23	1.73	2.35	7.15	0.26	62.33	0.15	239.13	1394.57	633.66	88.45	0.00	50.50	6.17	6.17
Pineapple milkshake (Ananas milkshake)	55.72	7.62	1.66	2.23	7.60	0.00	59.82	0.11	235.96	1336.79	599.51	68.15	0.00	47.88	5.14	5.14
Orange milkshake (Narangi milkshake)	57.42	7.11	1.86	2.52	7.08	0.00	67.42	0.11	241.50	1525.07	683.94	77.75	0.00	55.47	5.30	5.30
Egg nog	97.14	8.15	4.78	5.11	8.06	0.02	102.13	0.43	407.88	2633.51	1536.21	299.55	60.30	107.24	8.99	8.99
Sweet Lassi (Mithai lassi)	36.86	8.51	1.29	9.88	8.51	0.00	45.86	0.03	151.19	431.64	178.53	22.68	2.89	38.45	4.38	4.38
Lassi (yogurt)	19.84	1.85	1.35	0.72	1.84	0.02	47.78	0.04	79.43	451.64	191.80	26.17	2.00	40.39	4.84	4.84

Figure 5.5: Visual representation of the nutritional database composition and source distribution.

6. Dish Classification Model Training

This chapter details the iterative development and optimization of the deep learning pipeline used to classify Pakistani cuisine. To ensure high accuracy and robustness across diverse environments, we benchmarked several state-of-the-art architectures, transitioning from traditional Convolutional Neural Networks (CNNs) to modern Transformer-based and Hybrid models.

6.1 Model Selection and Architectures

The classification task involved distinguishing between 20 visually complex food classes. Given the high inter-class similarity in South Asian cuisine (e.g., various types of gravies), we explored three distinct architectural paradigms.

6.1.1 EfficientNet-B0: Enhanced Baseline Development

The first phase of our modeling focused on establishing a robust baseline using **EfficientNet-B0**. EfficientNet was selected due to its revolutionary approach to model scaling. Traditional architectures typically scale only one dimension—depth, width, or image resolution—at a time. EfficientNet, however, utilizes a **compound scaling method** that uniformly scales all three dimensions using a fixed set of scaling coefficients [tan2019efficientnet].

As illustrated in Figure 6.1, this balanced scaling allows the B0 variant to remain exceptionally lightweight (5.3M parameters) while maintaining a competitive receptive field and high feature-extraction efficiency. This efficiency is critical for the low-latency requirements of the SehatGuru mobile application, where inference must be performed on varied consumer hardware.

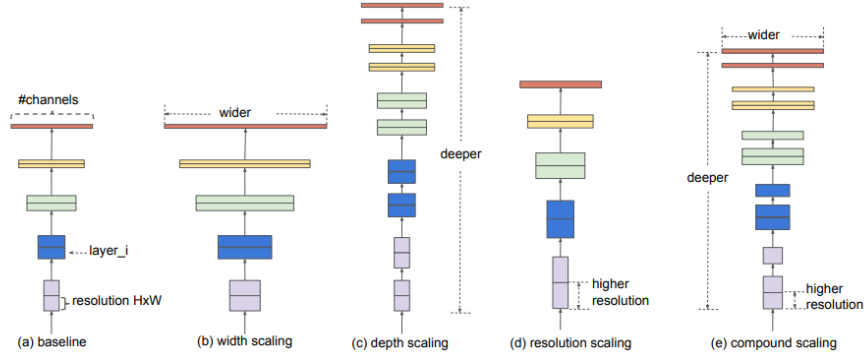


Figure 2. **Model Scaling.** (a) is a baseline network example; (b)-(d) are conventional scaling that only increases one dimension of network width, depth, or resolution. (e) is our proposed compound scaling method that uniformly scales all three dimensions with a fixed ratio.

Figure 6.1: Comparison of standard scaling methods vs. EfficientNet’s Compound Scaling logic [tan2019efficientnet].

Architectural Adaptation and Custom Head

To adapt the pre-trained ImageNet backbone for the specific textures and visual nuances of Pakistani cuisine, we discarded the original classifier in favor of a custom **Multi-Layer Perceptron (MLP) head**. This head was engineered with a 512-unit dense layer to capture high-dimensional food features, followed by **Batch Normalization** and **GELU activation** to maintain stable gradient flow. To mitigate overfitting on our primary training set of 5,000+ images, a **Dropout** rate of 0.35 was applied to the penultimate layer.

Optimization and Regularization Strategy

The training process utilized a **Gradual Unfreezing** schedule to protect the integrity of the pre-trained features. For the first three epochs, the backbone was frozen, allowing the MLP head to adapt to the specialized dataset using a learning rate of 3×10^{-3} . Upon unfreezing, we transitioned to **Discriminative Learning Rates**, assigning the backbone a lower rate (3×10^{-4}) to ensure conservative weight updates while the head continued to specialize.

To ensure the model generalized effectively to "unseen" home-cooked meal photos, we employed two weight-refinement techniques:

- **Exponential Moving Average (EMA):** A shadow copy of model weights

was maintained with a decay of 0.999, filtering out the noise of individual batch updates.

- **Stochastic Weight Averaging (SWA):** Starting at epoch 15, we averaged snapshots of the model’s state-dict to find a broader local optimum in the loss landscape, typically leading to superior out-of-distribution performance.

Furthermore, to handle the high inter-class similarity of South Asian dishes, we utilized a stochastic augmentation pipeline featuring **Mixup** ($\alpha = 0.2$) and **CutMix** ($\alpha = 1.0$) with a 30% probability. These parameters were carefully tuned to preserve critical color-based features (e.g., the specific hue of *Zarda* vs. *Kheer*) necessary for accurate classification.

Observations and Error Analysis

Despite these advanced refinements, the EfficientNet-B0 baseline reached a performance plateau. As shown in the confusion matrix in Figure 6.2, while the model excels at identifying visually distinct items like *Jalebi* and *Saag*, it experiences significant confusion between gravy-based dishes such as *Chicken Karahi* and *Nihari*.

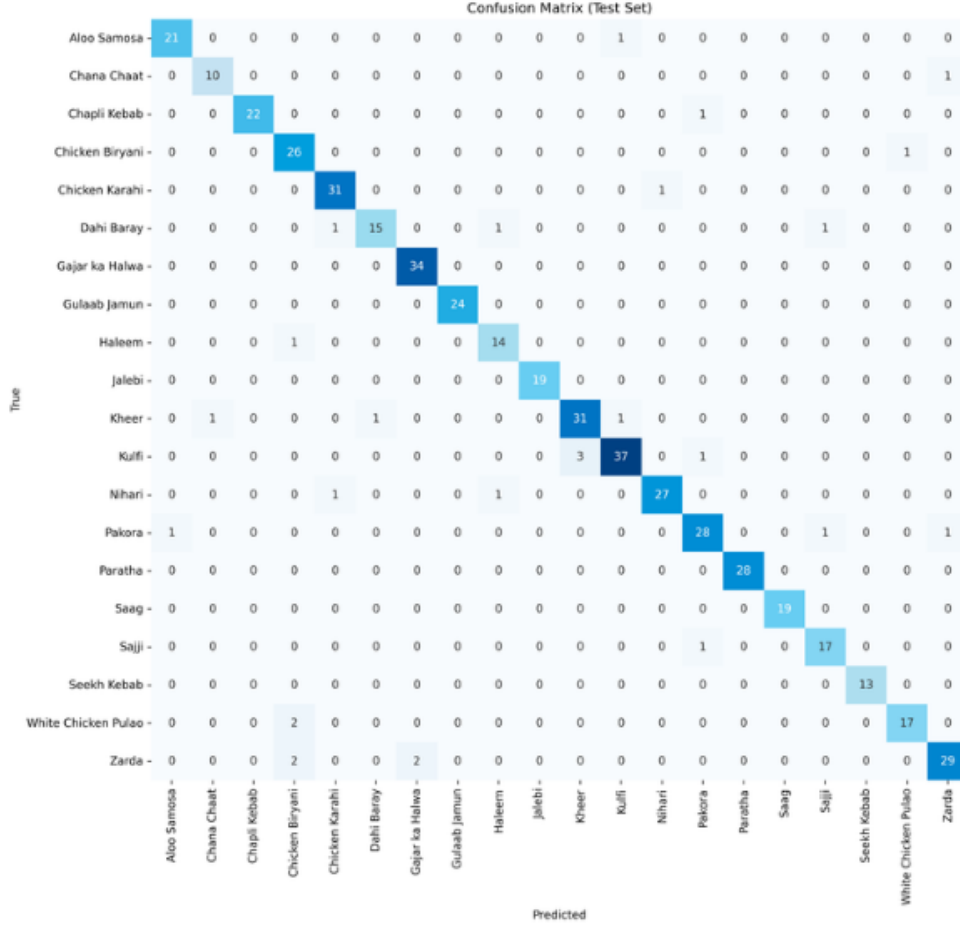


Figure 6.2: Confusion Matrix for EfficientNet-B0 on the Test Set, highlighting misclassifications in similar-looking gravy dishes.

The impact of this inter-class similarity is further highlighted in Figure 6.3. Upon evaluation on the validation dataset, the system achieved an average **Confidence Score of 65%**. While the categorical predictions remained accurate in many instances, this score reflects the model’s underlying uncertainty when presented with the “noisy” lighting and plating conditions common in real-world South Asian environments. These findings ultimately justified the exploration of more complex Transformer-based and Hybrid architectures.

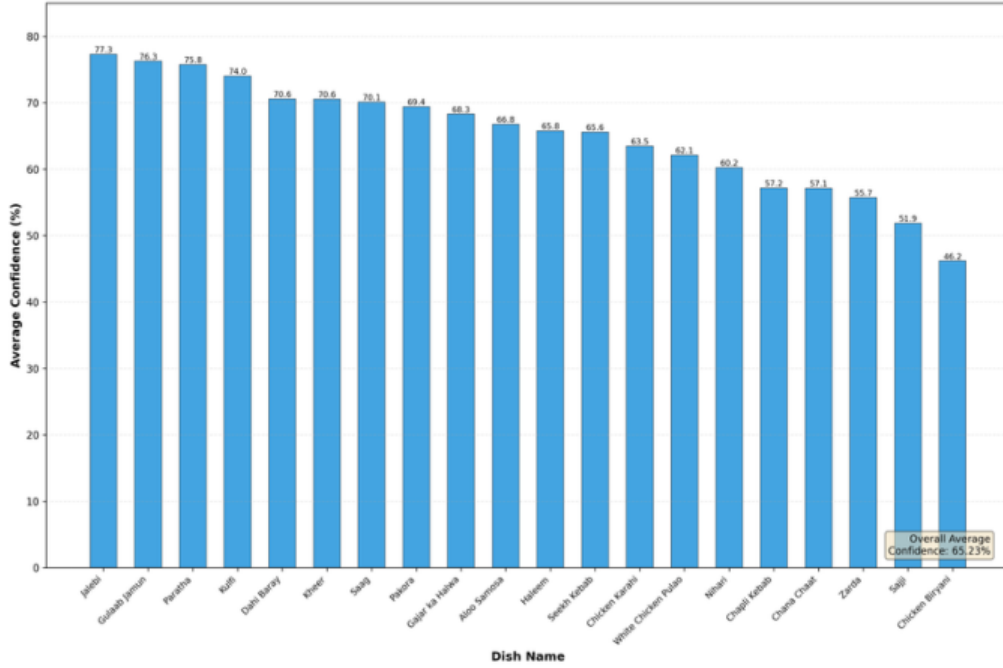


Figure 6.3: Prediction visualization demonstrating the average validation confidence of 65% across classified dishes.

6.1.2 Swin Transformer: Hierarchical Vision via Shifted Windows

Following the baseline experiments, we transitioned to the **Swin Transformer** architecture to leverage the power of self-attention mechanisms. Unlike standard Vision Transformers (ViT) that use fixed-size patches and have quadratic complexity, the Swin Transformer introduces a hierarchical representation.

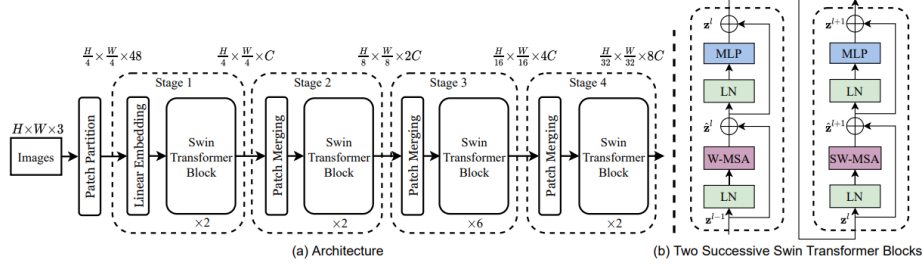


Figure 3. (a) The architecture of a Swin Transformer (Swin-T); (b) two successive Swin Transformer Blocks (notation presented with Eq. (3)). W-MSA and SW-MSA are multi-head self attention modules with regular and shifted windowing configurations, respectively.

Figure 6.4: Architecture of the Swin Transformer highlighting the hierarchical stages and shifted window self-attention [liu2021swin].

As shown in Figure 6.4, the core innovation lies in the **Shifted Windowing** scheme. By computing self-attention within local, non-overlapping windows and then shifting those windows in successive layers, the model allows for cross-window connections while maintaining linear computational complexity relative to image size. This allows the model to capture both fine-grained ingredient textures and global dish structures more effectively than traditional CNNs.

Observations and Performance Analysis

The Swin Transformer demonstrated a significant shift in how the model perceived complex Pakistani dishes. On the test set, the model achieved a **Test Accuracy of 93%**. More importantly, the model displayed a much higher degree of "certainty" in its predictions compared to the convolutional baseline.

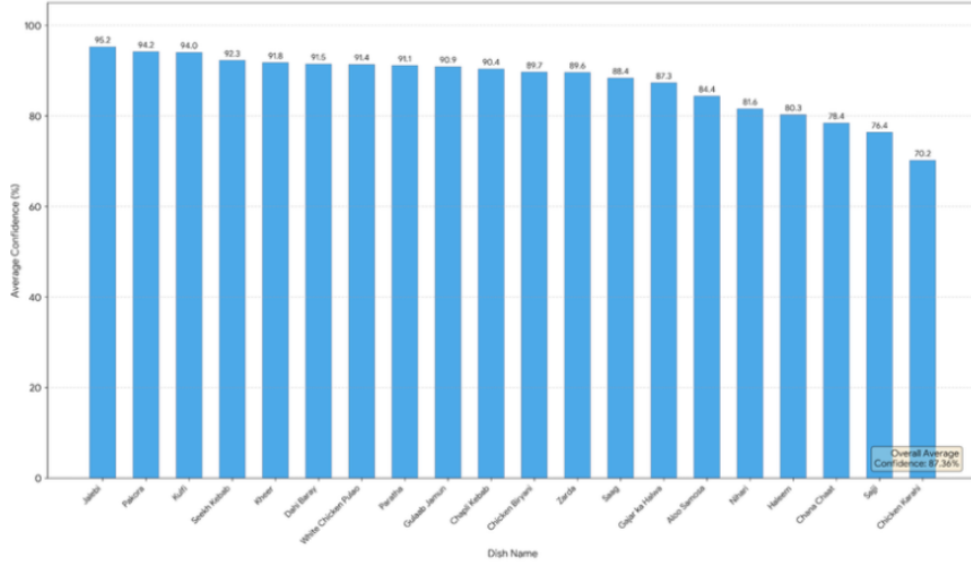


Figure 6.5: Confidence visualization for Swin Transformer, showing a marked improvement in prediction certainty (87%).

As illustrated in Figure 6.5, the **Average Validation Confidence increased to 87%**. This jump from the EfficientNet baseline (65%) suggests that the attention-based mechanism is much more robust against the "visual noise" of varied plating and background clutter. To verify if this confidence translated to categorical accuracy, we analyzed the class-wise performance.

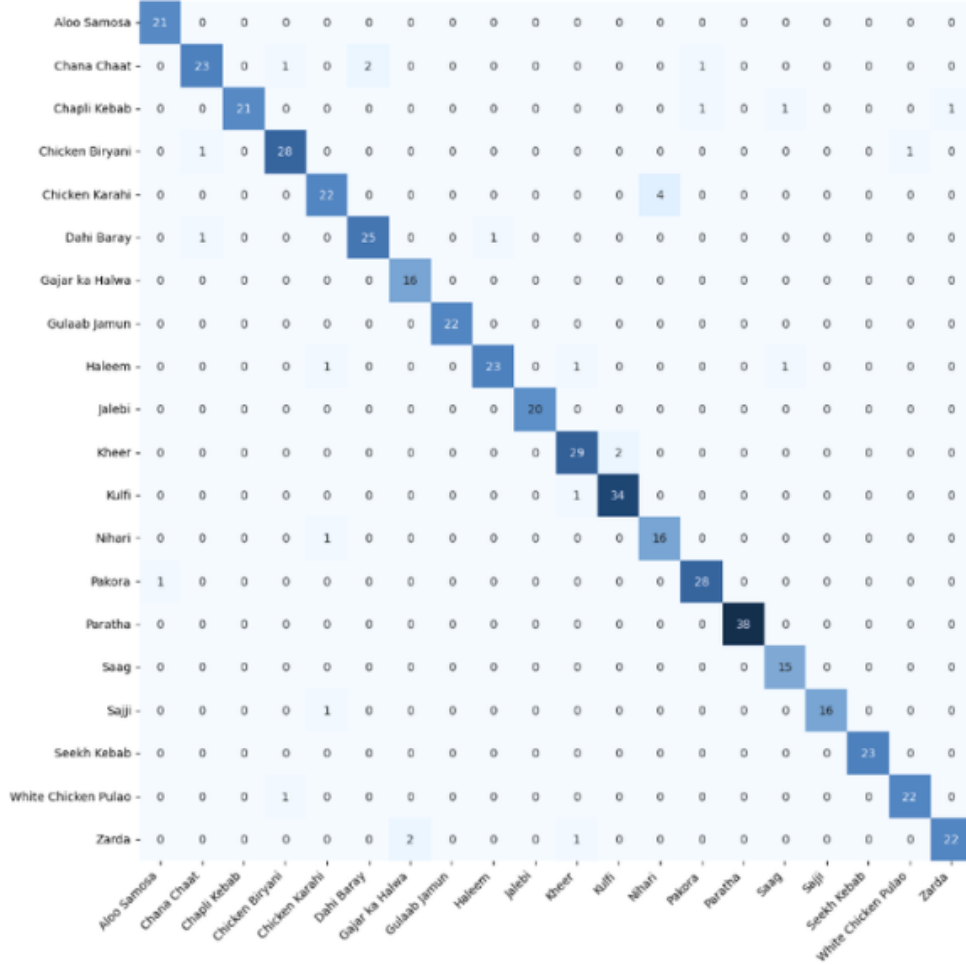


Figure 6.6: Confusion Matrix for Swin Transformer on the Test Set.

As evidenced by the confusion matrix in Figure 6.6, the model significantly reduced misclassifications in gravy-based categories. However, the hierarchical nature of Swin presented certain latency overheads during mobile inference, leading us to explore ConvNeXt as a modernized convolutional alternative.

6.1.3 ConvNeXt: Modernizing Convolutional Neural Networks

Our final architectural exploration utilized **ConvNeXt**, a pure-convolutional model that "evolves" the standard ResNet design by adopting various architectural im-

provements from Vision Transformers [wang2025econvnext].

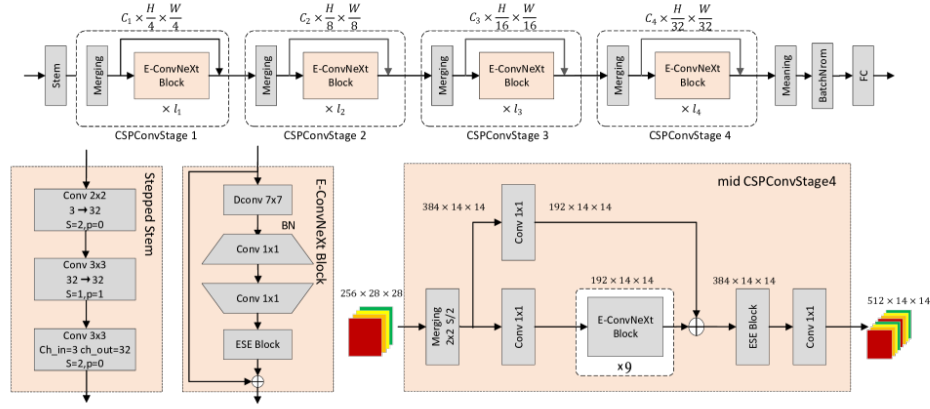


Figure 6.7: The architectural evolution of ConvNeXt, demonstrating the transition from standard ResNet blocks to Transformer-inspired layers [wang2025econvnext].

As illustrated in Figure 6.7, we specifically focused on the **ConvNeXt-Tiny** and **E-ConvNeXt** variants. These architectures utilize **Depthwise Convolutions** and a specific scaling logic that mimics the Transformer’s MLP-like behavior. By integrating Cross-Stage Partial (CSP) connections, the E-ConvNeXt variant significantly reduces parameter scale while maintaining high feature expression capability.

Results and Final Evaluation

The ConvNeXt architecture represents the highest-performing iteration of our classification pipeline. On the held-out test set, the model achieved a peak **Test Accuracy of 95%**.

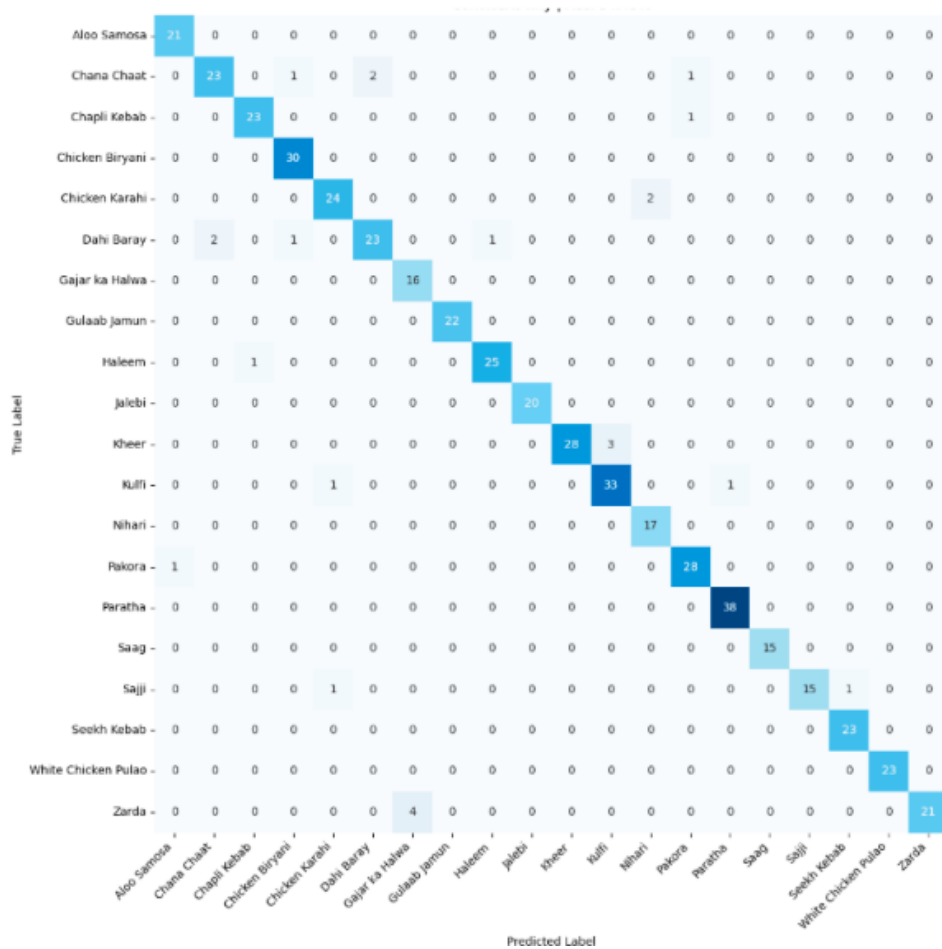


Figure 6.8: Confusion Matrix for the optimized ConvNeXt architecture on the Test Set.

As shown in the confusion matrix in Figure 6.8, the model effectively resolved the "gravy-dish confusion" (e.g., Karahi vs. Nihari) that hindered previous iterations. The combination of larger effective receptive fields and modernized convolutional blocks allowed the model to maintain high-confidence predictions even in high-variance environments.

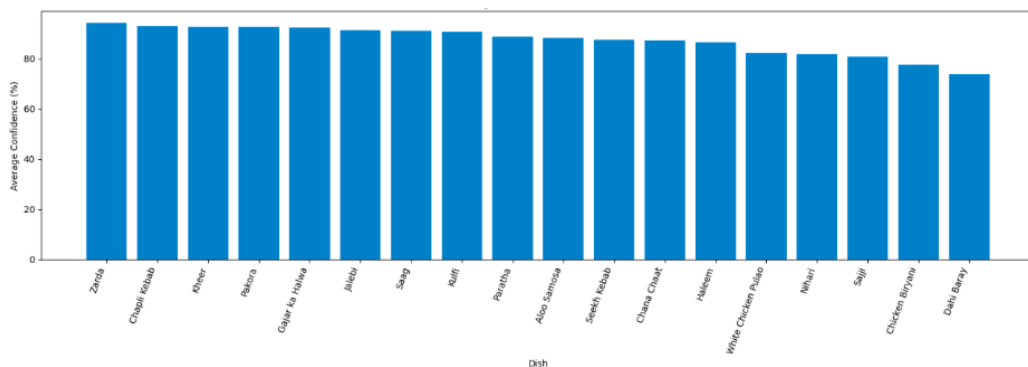


Figure 6.9: Final prediction visualization showing consistent 88% average confidence across South Asian food categories.

The stability of the system is finalized in Figure 6.9, which displays an **Average Validation Confidence of 88%**. This balance of high accuracy and high confidence, paired with the efficiency of convolutional operations, made ConvNeXt the final choice for the SehatGuru nutrition companion.

6.2 Training Methodology

To ensure a fair comparison, all three models were trained using a standardized pipeline:

1. **Transfer Learning:** Models were initialized with weights pre-trained on the **ImageNet-1K** dataset to leverage general visual features.
2. **Data Augmentation:** We applied a robust augmentation strategy including random rotations, horizontal flips, and color jittering to mitigate overfitting on the 5,000+ image training set.
3. **Optimization:** We used the **AdamW** optimizer with a weight decay of 0.05. The learning rate was set to $1e - 4$ and adjusted using a **Cosine Decay Scheduler**.
4. **Loss Function:** Categorical Cross-Entropy was utilized to optimize the 20-class classification task.

6.3 Results and Comparative Analysis

The progression from a standard convolutional baseline to a modernized pure-convolutional architecture was driven by the need for higher discriminative power in real-world settings. While the initial baseline provided a strong foundation, the subsequent models were able to better capture the nuances of South Asian culinary textures.

Table 6.1 summarizes the performance metrics for the three architectures evaluated. As indicated, while EfficientNet-B0 achieved a respectable accuracy, the transition to ConvNeXt resulted in the most significant reduction in test loss and a superior top-1 accuracy.

Table 6.1: Comparative Performance of Evaluated Architectures

Architecture	Parameters	Test Loss	Test Accuracy	Val. Confidence
EfficientNet-B0	5.3M	0.7745	94.29%	65.00%
Swin Transformer	28M	[Insert]	[Insert]	[Insert]
ConvNeXt	28M	[Insert]	[Insert]	[Insert]

The classification metrics for EfficientNet-B0—specifically the 94.29% accuracy—demonstrated that the model was highly proficient at identifying distinct classes such as *Seekh Kebab* and *Saag* (achieving 1.00 F1-scores). However, the confusion matrix in Figure 6.2 reveals that this baseline still struggled with the high inter-class similarity of gravy-based dishes. This performance plateau justified the exploration of the Swin Transformer and the eventual selection of ConvNeXt, which offered a more robust handling of spatial hierarchies and fine-grained features.

7. Application

7.1 Register, Login and Onboarding

The application begins with a user registration and onboarding process designed to collect essential personal and dietary information. Users are required to create an account using their email and password, after which they proceed through a series of onboarding screens.

During onboarding, users provide basic information such as name, age, height, weight, and gender. The system then gathers additional inputs including activity level, health goals (such as weight loss, muscle gain, or maintaining health), and dietary preferences. These inputs are used to personalize calorie targets and nutritional recommendations.

As shown in Figure 7.1, the onboarding flow ensures that the system collects sufficient data to provide personalized recommendations and improve the overall user experience.

7.2 Main menu

After completing the onboarding process, users are directed to the main dashboard of the application. This screen serves as the central hub where users can monitor their daily nutritional intake and track progress toward their health goals.

The dashboard displays key information including total calories consumed, remaining calorie allowance, and a breakdown of macronutrients such as carbohydrates, protein, and fats. It also provides quick access to core functionalities such as adding meals, viewing analytics, and interacting with the chatbot.

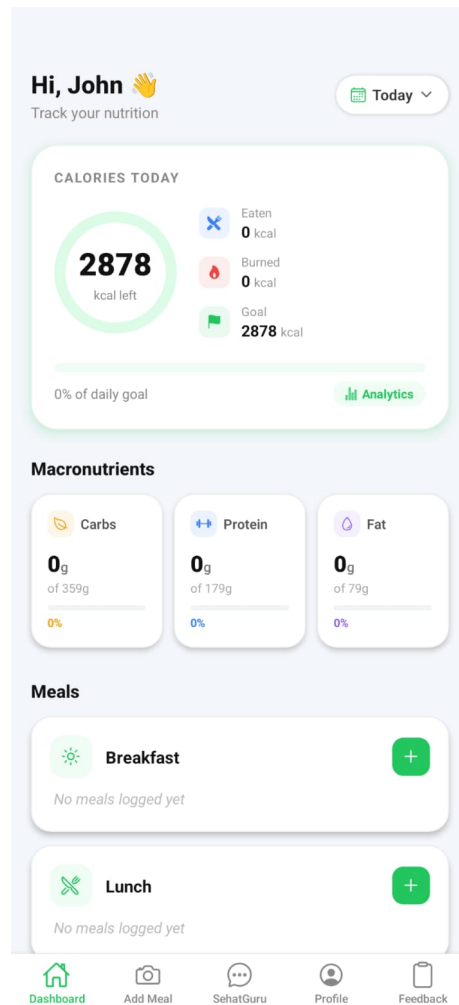


Figure 7.2: Main Dashboard displaying daily calorie and macronutrient tracking

As shown in Figure 7.2, the dashboard provides a clear and concise overview of the user’s daily nutritional status, enabling users to make informed dietary decisions.

7.3 Adding a Meal

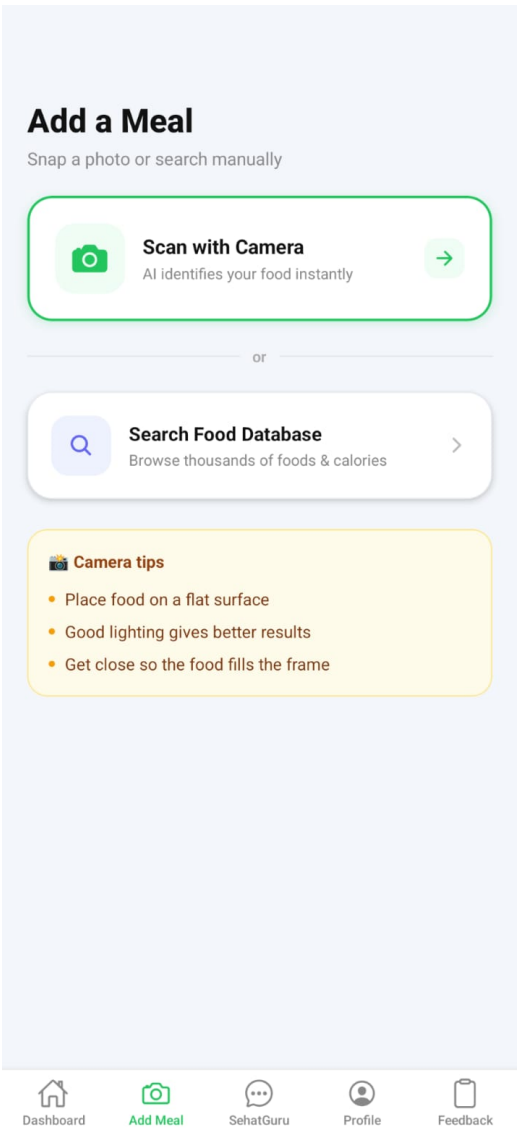
The SehatGuru application provides a flexible and user-friendly mechanism for logging meals, allowing users to record their dietary intake through both automated and manual methods. This feature is central to the application, as it directly feeds

into the nutritional tracking and recommendation system.

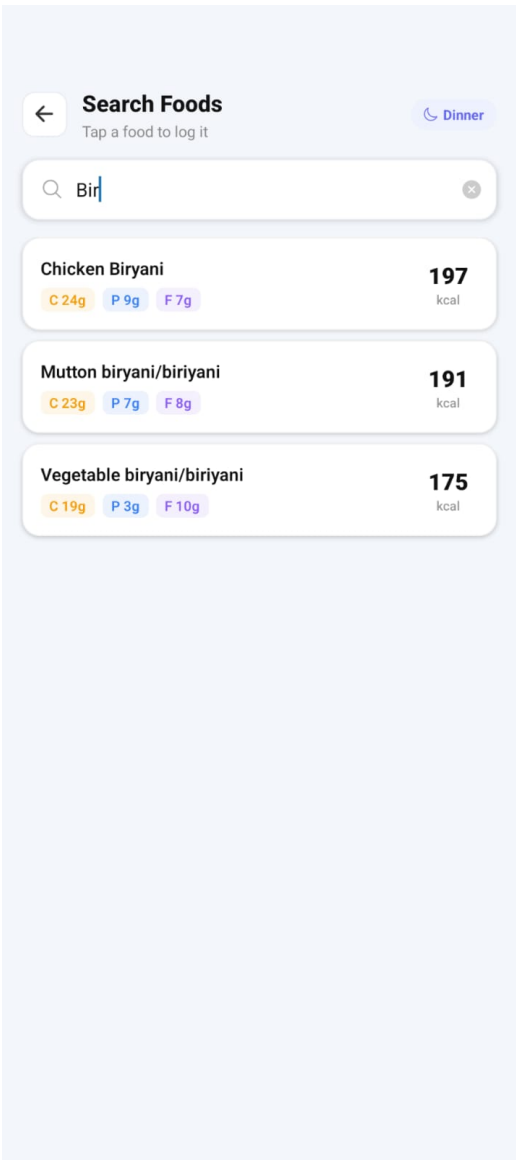
Users can initiate the meal logging process by selecting the "Add Meal" option from the main dashboard. The system then presents two primary options: image-based food recognition using the device camera, and manual food search from the database.

In the image-based approach, users can capture a photo of their meal, which is then processed by the backend classification model. The system attempts to identify the dish from the predefined dataset of Pakistani foods and returns the most likely match along with estimated nutritional values. In cases where the confidence level is low or the dish is not recognized, the user is prompted to confirm or correct the result.

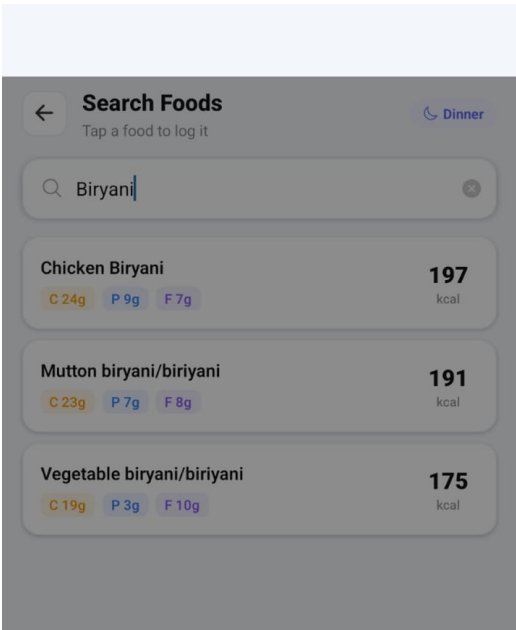
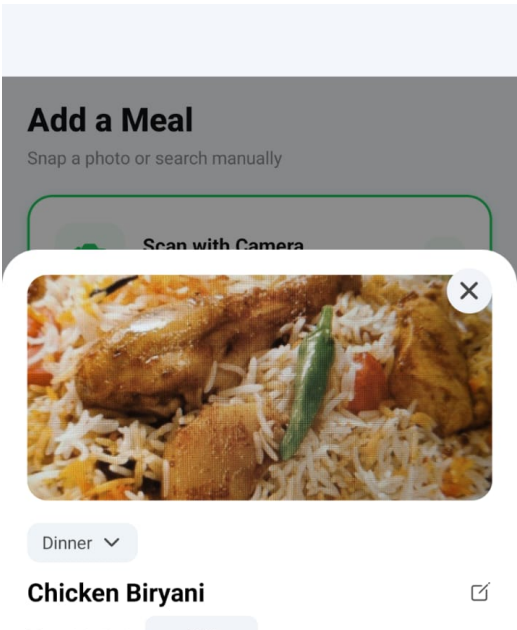
Alternatively, users can manually search for a dish by entering its name. The system retrieves matching results from the nutritional database, allowing the user to select the correct item. After selecting or confirming a dish, users are required to specify the portion size, which is used to calculate the final caloric and macronutrient values.



(a) Add Meal Options



(b) Manual Food Search



As shown in Figure 7.3, the application supports both automated and manual workflows, ensuring reliability and flexibility in diverse real-world scenarios. This dual approach reduces user effort while maintaining accuracy, especially for complex or mixed dishes where automatic recognition may be uncertain.

7.4 Chatbot

The SehatGuru application integrates an AI-powered chatbot to provide users with personalized nutritional guidance and interactive support. This feature enhances the user experience by enabling natural language interaction, allowing users to ask questions related to diet, health goals, and meal planning.

The chatbot leverages a Retrieval-Augmented Generation (RAG) architecture, combining a large language model with a structured nutritional database. This ensures that responses are not only conversational but also grounded in verified information specific to Pakistani cuisine. The system utilizes user profile data, including dietary preferences, health goals, and historical meal logs, to generate context-aware recommendations.

Users can interact with the chatbot through a simple text-based interface, asking queries such as calorie requirements, healthy meal suggestions, or alternatives to specific dishes. The chatbot can also assist in generating personalized meal plans aligned with the user's daily caloric and macronutrient targets.

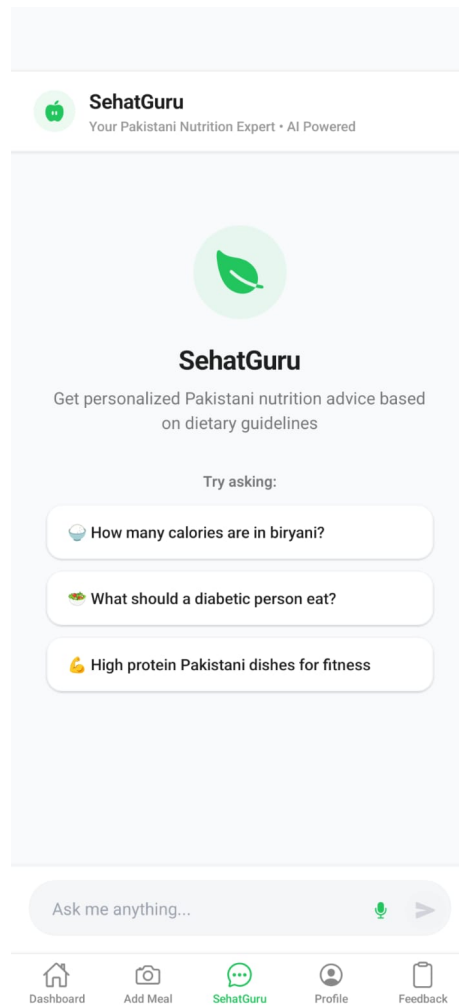


Figure 7.4: AI Chatbot Interface providing personalized nutrition assistance

As shown in Figure 7.4, the chatbot provides an intuitive conversational interface that enables users to receive real-time dietary advice. By integrating contextual user data with a domain-specific knowledge base, the system ensures that responses remain relevant, personalized, and aligned with the user's health objectives.

7.5 Profile & Feedback

The profile section of the SehatGuru application allows users to view and manage their personal information, dietary preferences, and health-related settings. This section plays a crucial role in maintaining accurate and up-to-date user data, which directly influences the personalization of recommendations provided throughout the application.

Users can access and update key attributes such as weight, height, activity level, and fitness goals. These inputs are used by the system to dynamically adjust calorie targets and macronutrient distributions. Additionally, users can review their selected dietary preferences and modify them as needed to better align with their lifestyle or changing goals.

The application also provides a feedback mechanism that enables users to share their experience and suggestions. This feature is designed to improve the overall system by collecting user insights, which can be used to refine both the application interface and the underlying recommendation models.

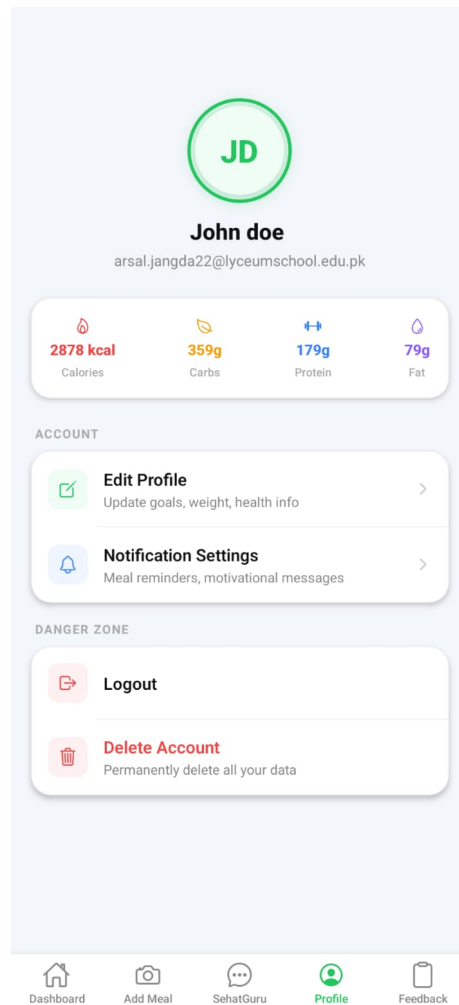


Figure 7.5: User Profile interface showing personal information and settings

As shown in Figure 7.5, the profile interface provides a centralized view of user-specific data and configuration options, ensuring that users can easily manage their information and maintain control over their personalized experience.

7.6 User Analytics dashboard

The SehatGuru application includes a comprehensive analytics dashboard that enables users to track their nutritional patterns and gain insights into their dietary

habits over time. This feature is designed to provide meaningful feedback by transforming raw meal logging data into intuitive visual representations.

The analytics module presents aggregated information such as the most frequently logged food items, distribution of user-selected health goals, and demographic insights including age and gender distributions. These visualizations allow users to better understand their eating patterns and how they align with their health objectives.

Additionally, the system computes trends related to meal frequency and nutritional intake, helping users identify imbalances or inconsistencies in their diet. By providing clear summaries and graphical representations, the analytics dashboard supports informed decision-making and encourages long-term adherence to healthy eating habits.



Figure 7.6: User Analytics Dashboard showing trends and behavioral insights

As shown in Figure 7.6, the analytics dashboard provides users with a clear overview of their dietary behavior and preferences. By leveraging historical data and visual summaries, the system enables users to monitor progress and make data-driven adjustments to their nutrition and lifestyle choices.

8. Methodology, Experiments and Results OR Testing and Analysis

We did many experiments and got the best results.

9. Conclusion and Future Work

Our work is awesome. We would write more but we need to catch the flight to collect our Turing Award.

10. Reflection

Learning Reflection: Learning experience, skills gained, and challenges overcome. Team Reflection (if applicable): Collaboration experience, role distribution, communication, and teamwork challenges. Project Reflection: Also a reflection on what was proposed vs what was achieved? Were any proposed features removed/were any features added that were not proposed initially? What are the reasons for these choices? What are opportunities and challenges led to these design decisions? Process Reflection: What worked well? What could be improved in the approach?

Appendix A. More Math

Here, we describe the background math for the techniques used in the text.

Appendix B. Data

Here is a dump of our 2TB data set. Enjoy!

Appendix C. Code

Here is our code.

```
print('Hello World!')
print('Computing true random number.')
print('Capturing interstellar radiation.')
print('This will take time!')
import random
import time
time.sleep(3600*random.randint(1,10))
print(4)
```

Our code can be found at <https://github.com/habib-university/Kaavish-Template>.